

HASH-BASED SIGNATURES

1. INTRODUCTION

Alfred Menezes
cryptography101.ca

© Alfred Menezes
May 2025

1. Signature schemes
2. Security definition
3. Families of signature schemes
4. Hash-based signature schemes
5. Roadmap

1. Introduction

2

Hash-based signatures © Alfred Menezes

Signature schemes

- ♦ Signature schemes are used for **authentication**
 - ♦ data integrity and data origin authentication.
- ♦ A signature scheme has three components:
 1. **Key generation:** produces a public/private key pair.
 2. **Signature generation:** uses the (*signing*) private key to produce a signature for a message.
 3. **Signature verification:** uses the (*verification*) public key to decide whether the signed message is genuine or not.



Security definition

- ♦ **Definition:** A signature scheme is said to be *secure* if it is existentially unforgeable by a computationally bounded adversary who launches a chosen-message attack.

- ♦ **Note:** The adversary has access to a *signing oracle*, from which she can obtain the signatures of any messages of her choosing. The adversary's goal is to produce a single valid message-signature pair for any message (of the adversary's choosing) that was not previously given to the signing oracle.

Families of signature schemes

1. RSA

- ◆ Completely broken by large-scale quantum computers

2. Elliptic curve cryptography (ECDSA, EdDSA)

- ◆ Completely broken by large-scale quantum computers

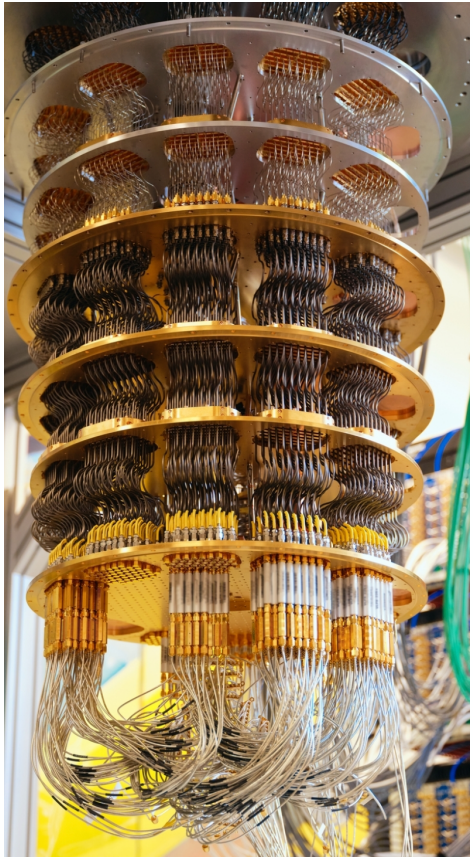
3. Lattice-based cryptography (Dilithium, Falcon)

- ◆ Quantum safe, but security relies on relatively new lattice problems

Note: All three families of signature schemes also rely on security properties of hash functions.

Hash-based signature schemes

- ◆ Hash-based signature schemes were first proposed by **Leslie Lamport** in 1976.
- ◆ Improvements by **Robert Winternitz** and **Ralph Merkle** in 1979.
- ◆ The only cryptographic primitive employed is a hash function.
- ◆ Main advantage: Security is well understood, *against both classical and quantum attacks*.
- ◆ Main drawbacks: Large public keys and large signatures.



Standardized hash-based signature schemes

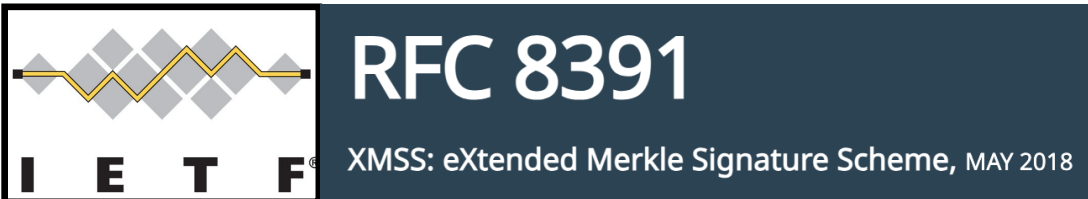
1. Leighton-Micali Signature scheme (LMS):

- ◆ NIST SP 800-208, IETF RFC 8554

2. eXtended Merkle Signature Scheme (XMSS):

- ◆ NIST SP 800-208, IETF RFC 8391

3. SPHINCS+: FIPS 205



Roadmap

- ◆ V1: Introduction
- ◆ V2: Hash functions
- ◆ V3: Lamport signature scheme
- ◆ V4: Lamport signatures: problems and solutions
- ◆ V5: Leighton-Micali signature scheme
- ◆ V6: SPHINCS+

HASH-BASED SIGNATURES

2. HASH FUNCTIONS

Alfred Menezes
cryptography101.ca

- 1. Hash functions
- 2. Security properties
- 3. Classical attacks
- 4. Quantum attacks
- 5. SHA-2 and SHA-3

© Alfred Menezes
May 2025

Hash functions

A **hash function** is a mapping H such that:

- 1. H maps binary messages of arbitrary lengths $\leq L$ to outputs of a fixed length n :
 $H : \{0,1\}^{\leq L} \rightarrow \{0,1\}^n$. (L is usually large, e.g., $L = 2^{64}$, whereas n is small, e.g. $n = 256$.)
- 2. $H(x)$ can be efficiently computed for all $x \in \{0,1\}^{\leq L}$.

- ♦ H is called an **n -bit hash function**. $H(x)$ is called the **hash** or **message digest** of x .
- ♦ Notes:
 - ♦ The description of a hash function is public; there are no secret keys.
 - ♦ For simplicity, we usually write $\{0,1\}^*$ instead of $\{0,1\}^{\leq L}$.
 - ♦ More generally, a hash function $H : S \rightarrow T$ is an efficiently computable function from a set S to a set T .

Security properties

Definition: A hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is **preimage resistant** (PR) if, given a hash value $y \in_R \{0,1\}^n$, it is computationally infeasible to find (with non-negligible success probability) *any* $x \in \{0,1\}^*$ with $H(x) = y$.

Definition: A hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is **2nd preimage resistant** (2PR) if, given $x \in_R \{0,1\}^*$, it is computationally infeasible to find (with non-negligible success probability) *any* $x' \in \{0,1\}^*$ with $x' \neq x$ and $H(x') = H(x)$.

Definition: A hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is **collision resistant** (CR) if it is computationally infeasible to find (with non-negligible success probability) *any* $x, x' \in \{0,1\}^*$ with $x' \neq x$ and $H(x') = H(x)$.

Breaking PR, 2PR, CR

Breaking PR:

Given: $y \in_R \{0,1\}^n$.

Required: $x \in \{0,1\}^*$ with $H(x) = y$.

$$H : \{0,1\}^* \longrightarrow \{0,1\}^n$$

Breaking 2PR:

Given: $x \in_R \{0,1\}^*$.

Required: $x' \in \{0,1\}^*$ with $x' \neq x$ and $H(x') = H(x)$.

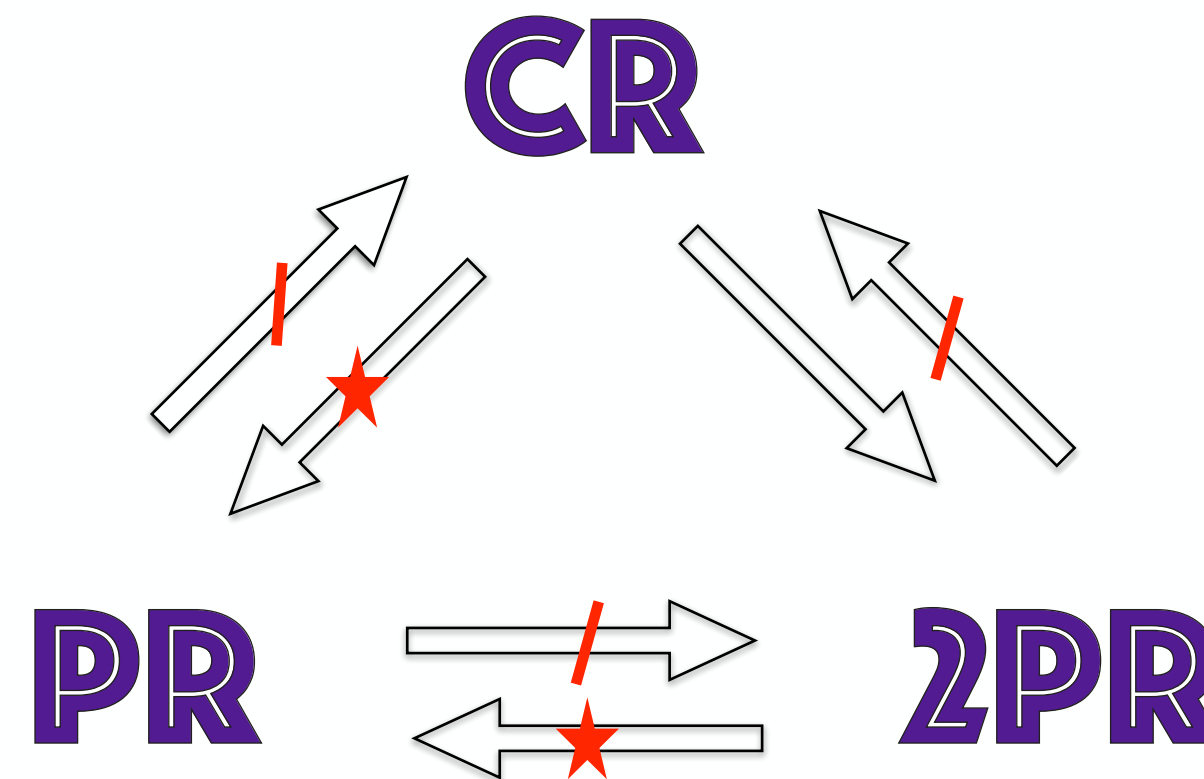
Breaking CR:

Given: —.

Required: $x, x' \in \{0,1\}^*$ with $x' \neq x$ and $H(x') = H(x)$.

Relationships between security properties

Let $H : \{0,1\}^* \longrightarrow \{0,1\}^n$ be a hash function.



See V3b of
Crypto 101: Building Blocks

★ for somewhat uniform
hash functions

Classical attacks

A **generic attack** on hash functions $H : \{0,1\}^* \longrightarrow \{0,1\}^n$ does not exploit any properties that the specific hash function might have.

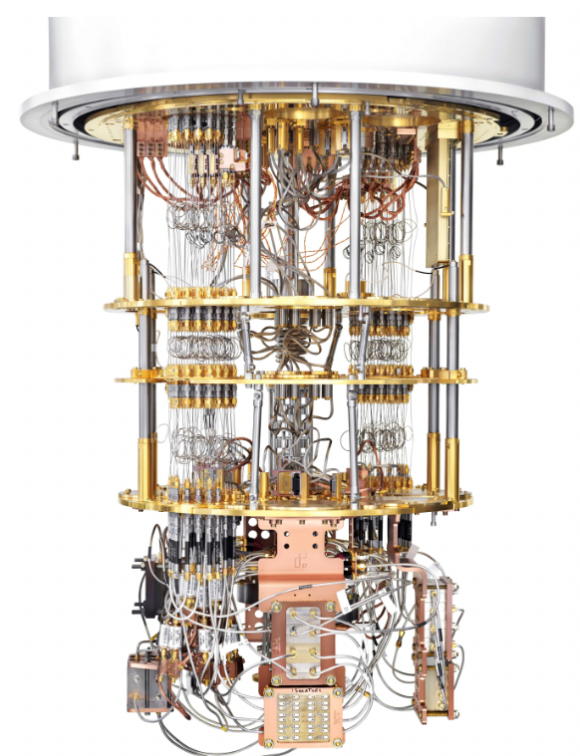
Running times of fastest generic attacks for:

- ♦ finding **preimages**: 2^n .
- ♦ finding **second-preimages**: 2^n .
- ♦ finding **collisions**: $2^{n/2}$ (VW parallel collision search)
[see V3c of “Crypto 101: Building Blocks”].

Quantum attacks

Running times of fastest/most cost-effective generic quantum attacks for:

- ♦ finding **preimages**: $2^{n/2}$ [Grover’s search, V1a of “Kyber and Dilithium”].
- ♦ finding **second-preimages**: $2^{n/2}$ [Grover’s search].
- ♦ finding **collisions**: $2^{n/2}$ [VW parallel collision search].



SHA-2 and SHA-3

- ✦ SHA-2 family: SHA-256, SHA-384, SHA-512 [iterated].
- ✦ SHA-3 family (Keccak): SHA3-256, SHA3-384, SHA3-512 [sponge].

Hash function	Preimage resistance	Second-preimage resistance	Collision resistance
SHA-256/ SHA3-256	128	128	128
SHA-384/ SHA3-384	192	192	192
SHA-512/ SHA3-512	256	256	256

Security level (in bits)
of SHA-2 and SHA-3
hash functions against
quantum attacks

HASH-BASED SIGNATURES

3. THE LAMPORT SIGNATURE SCHEME

Alfred Menezes
cryptography101.ca

© Alfred Menezes
May 2025

Outline

1. Signing a single bit
2. Lamport signature scheme
3. Security
4. Efficiency

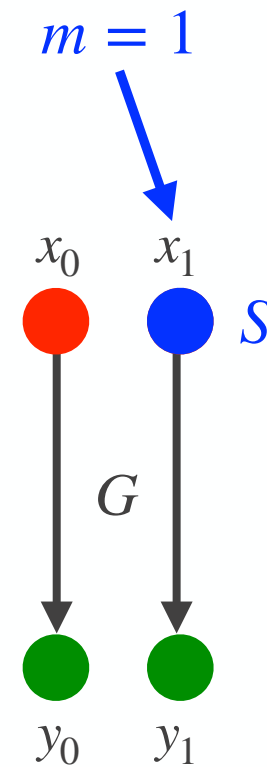
Introduction

- ✦ Leslie Lamport described his signature scheme to Whit Diffie in 1975.
- ✦ It appeared in the landmark 1976 Diffie-Hellman paper “New directions in public-key cryptography”.
- ✦ Lamport’s scheme is presented in a 1979 technical report “Constructing digital signatures from a one way function”.
- ✦ The security of Lamport’s signature scheme, *against both classical and quantum attacks*, is based only on standard security requirements for hash functions, and not on any number-theoretic assumption.



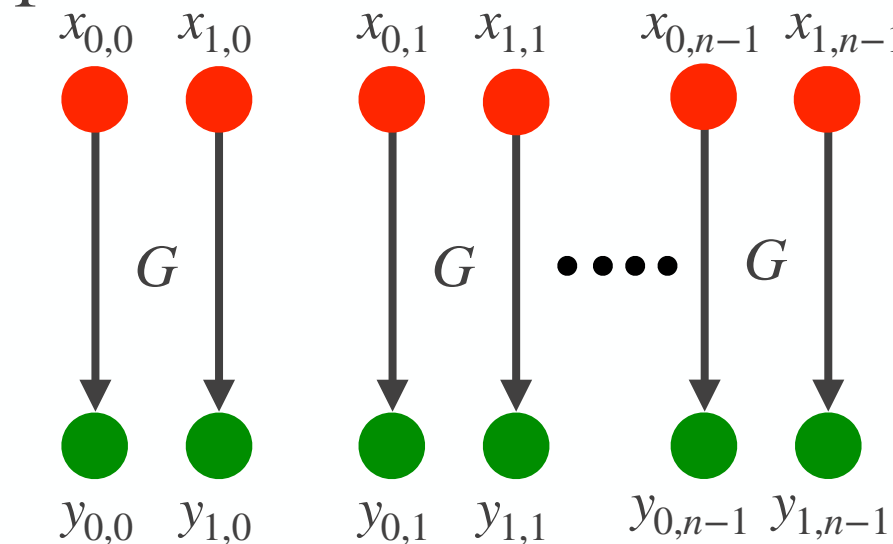
Signing a single bit

- ♦ $G : \{0,1\}^n \rightarrow \{0,1\}^n$ is a preimage-resistant hash function.
- ♦ **Key generation:** Alice selects $x_0, x_1 \in_R \{0,1\}^n$, and computes $y_0 = G(x_0)$, $y_1 = G(x_1)$.
Her private key is $X = (x_0, x_1)$.
Her public key is $Y = (y_0, y_1)$.
- ♦ **Signature generation:** To sign a bit $m \in \{0,1\}$, Alice sets $S = x_m$. Her signed message is (m, S) .
- ♦ **Signature verification:** Anyone who possesses an authentic copy of Y can verify that $y_m = G(S)$.



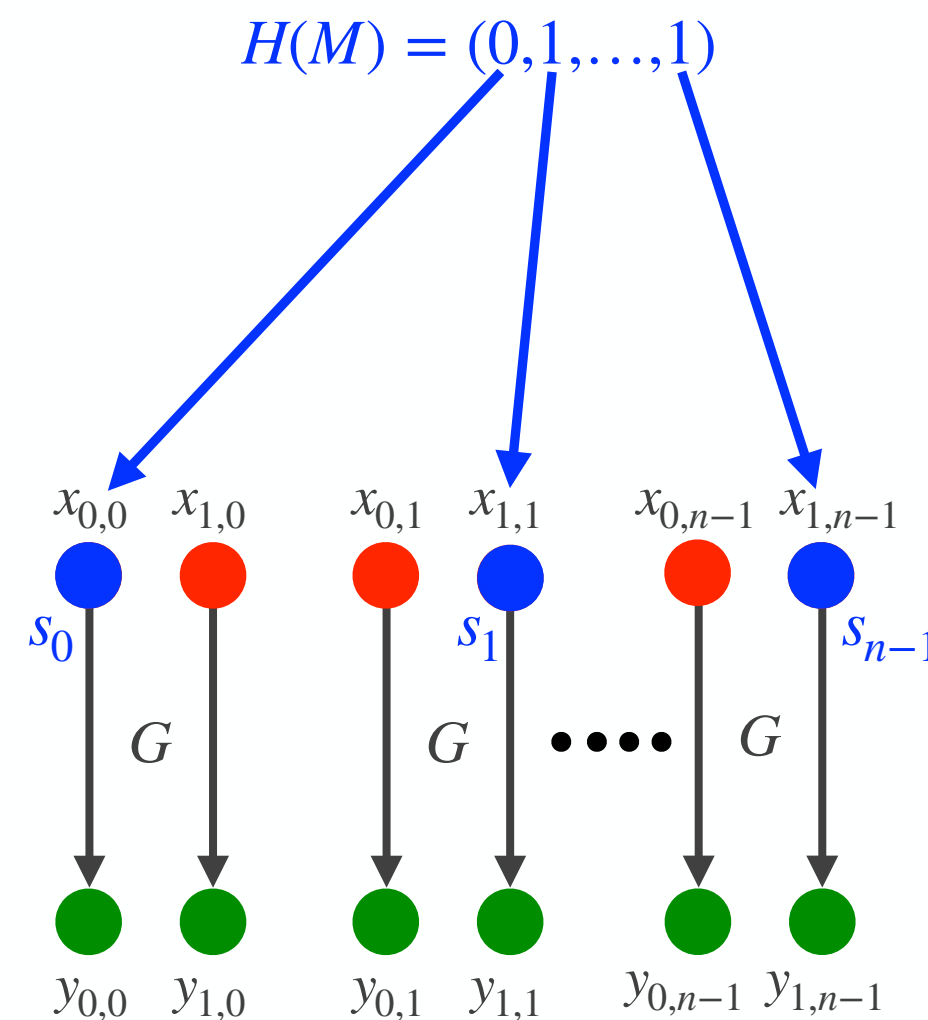
Lamport signature scheme: key generation

- ♦ $G : \{0,1\}^n \rightarrow \{0,1\}^n$ is a preimage-resistant hash function.
- ♦ $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is a collision-resistant hash function.
- ♦ **Key generation:** Alice does the following:
 - ♦ Select $x_{i,j} \in_R \{0,1\}^n$ for $i = 0,1$, $j = 0,1,\dots,n-1$
 - ♦ Compute $y_{i,j} = G(x_{i,j})$.
 - ♦ Alice's **private key** is $X = (x_{i,j})$.
 - ♦ Alice's **public key** is $Y = (y_{i,j})$.



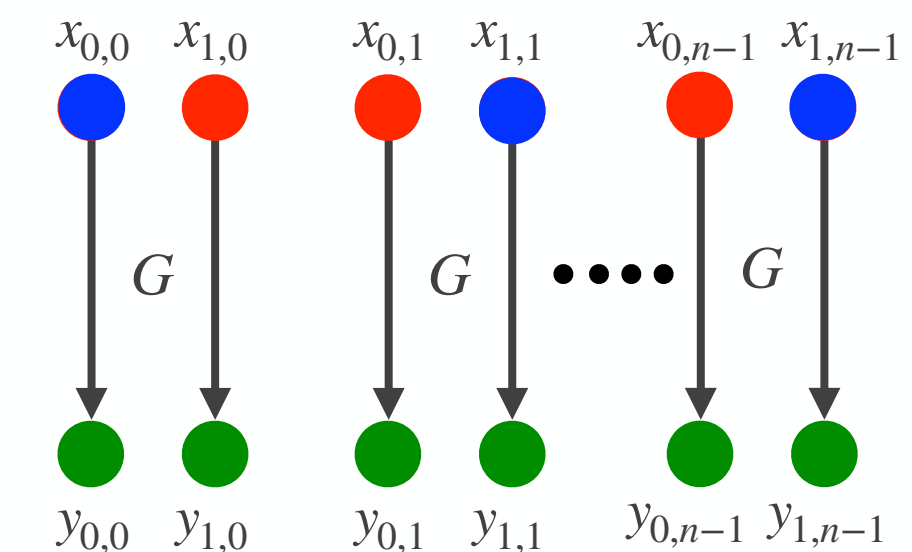
Lamport: signature generation/verification

- ♦ **Signature generation:** To sign $M \in \{0,1\}^*$, Alice computes $h = H(M) = (h_0, h_1, \dots, h_{n-1})$. Her signature on M is $S = (s_0, s_1, \dots, s_{n-1})$, where $s_j = x_{h_j,j}$.
- ♦ **Signature verification:** To verify (M, S) , anyone who possesses an authentic copy of Y can compute $h = H(M)$ and check that $y_{h_j,j} = G(s_j)$ for $j = 0,1,\dots,n-1$.



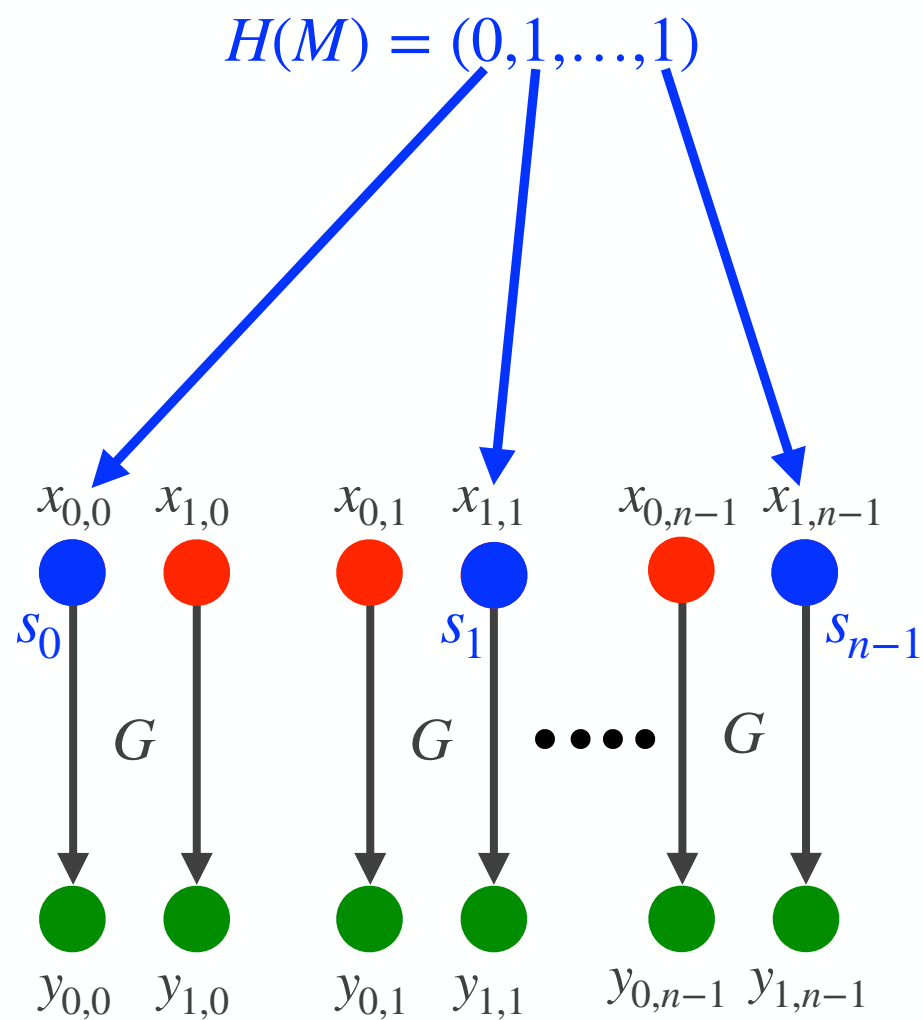
One-time signature scheme (OTS)

- ♦ A signed message (M, S) exposes half the components of Alice's private key X .
- ♦ A second signed message (M', S') is expected to expose an additional one-quarter of Alice's private key components.
- ♦ Thus, to preserve security, Alice should sign only one message using the same private key.
- ♦ So, Lamport's signature scheme is classified as a **one-time signature scheme (OTS)**.

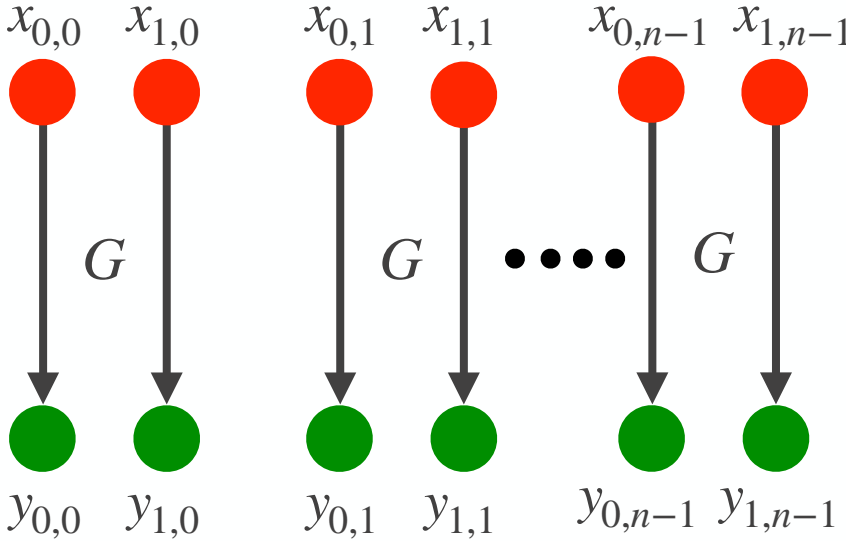


♦ **Definition:** A one-time signature scheme is said to be *secure* if it is existentially unforgeable by a computationally bounded adversary who is permitted to obtain the signature for one message of the adversary's choosing.

♦ **Claim:** If H is collision resistant and G is preimage resistant, then Lamport's signature scheme is a secure OTS.



- ♦ Private key and public key sizes: $2n^2$ bits.
 - ♦ For $n = 256$, this is 16,384 bytes.
- ♦ Signature size: n^2 bits.
 - ♦ For $n = 256$, this is 8,192 bytes.
- ♦ Key generation time: $2n$ G -evaluations.
- ♦ Signature generation: 1 H -evaluation.
- ♦ Signature verification: 1 H -evaluation and n G -evaluations.



HASH-BASED SIGNATURES

4. LAMPORT: PROBLEMS AND SOLUTIONS

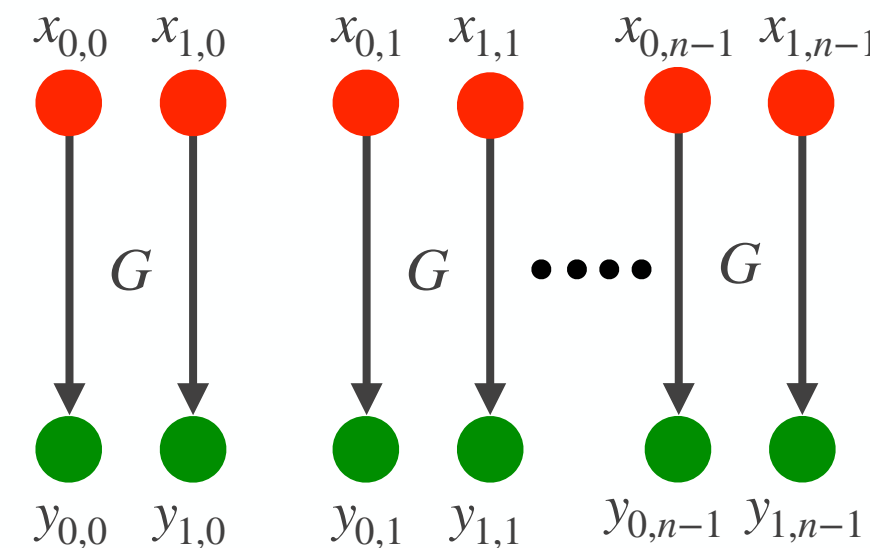
Alfred Menezes
cryptography101.ca

Outline

1. Lamport drawbacks: Large public keys, private keys, and signatures
2. Winternitz OTS
3. Merkle trees
4. Hypertrees

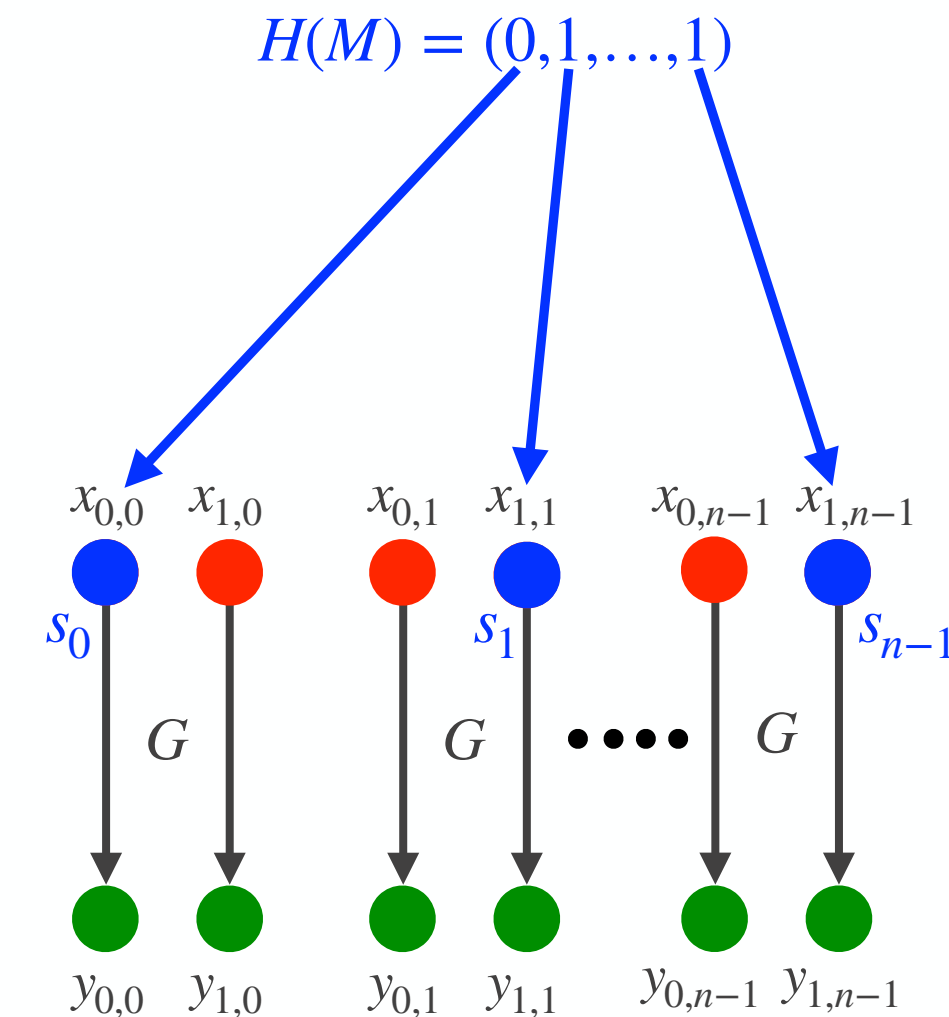
Lamport OTS: key generation

- ♦ $G : \{0,1\}^n \rightarrow \{0,1\}^n$ is a PR hash function.
- ♦ $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is a CR hash function.
- ♦ **Key generation:** Alice does the following:
 - ♦ Select $x_{i,j} \in_R \{0,1\}^n$ for $i = 0,1, \dots, n-1$, $j = 0,1, \dots, n-1$
 - ♦ Compute $y_{i,j} = G(x_{i,j})$.
 - ♦ Alice's **private key** is $X = (x_{i,j})$.
 - ♦ Alice's **public key** is $Y = (y_{i,j})$.



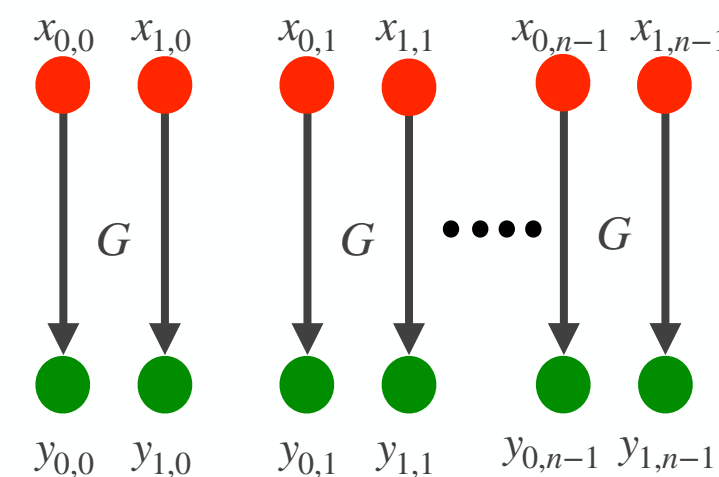
Lamport OTS: signature generation/verification

- ♦ **Signature generation:** To sign $M \in \{0,1\}^*$, Alice computes $h = H(M) = (h_0, h_1, \dots, h_{n-1})$. Her signature on M is $S = (s_0, s_1, \dots, s_{n-1})$, where $s_j = x_{h_j,j}$.
- ♦ **Signature verification:** To verify (M, S) , anyone who possesses an authentic copy of Y can compute $h = H(M)$ and check that $y_{h_j,j} = G(s_j)$ for $j = 0,1, \dots, n-1$.



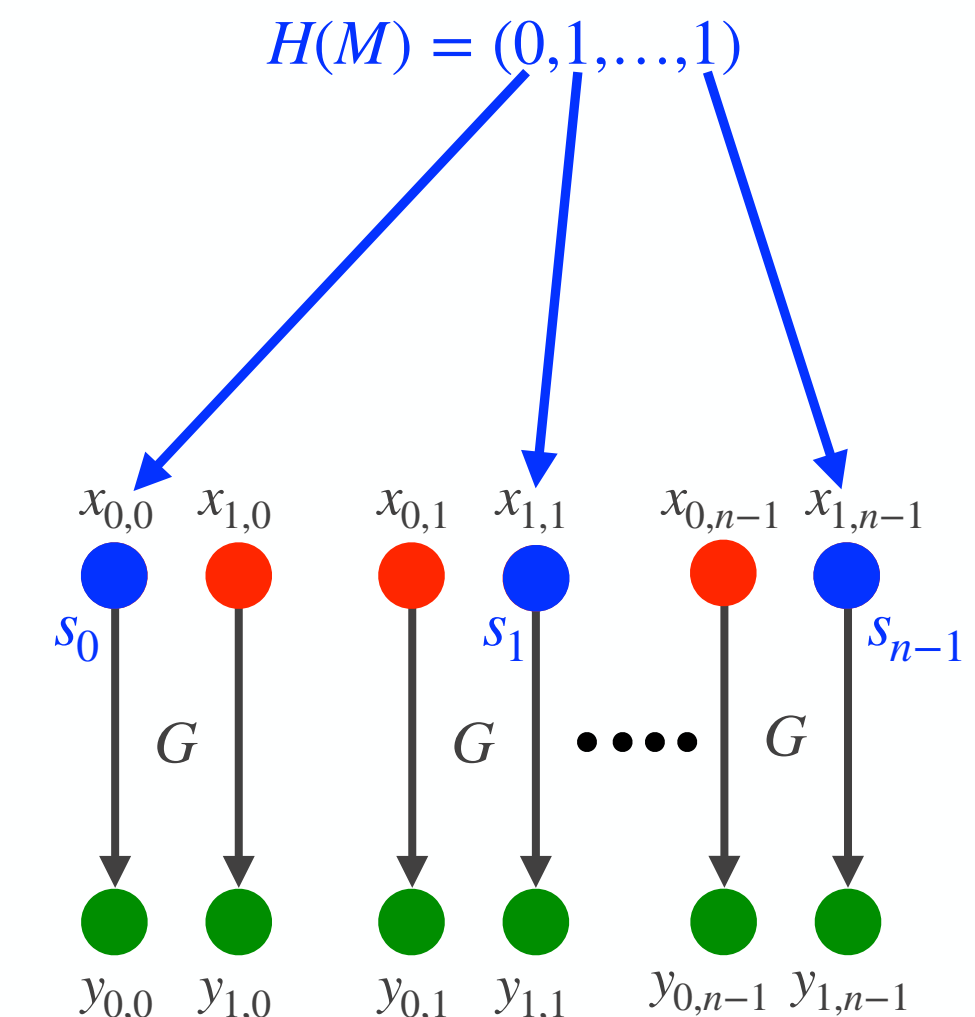
Problem #1: Large private keys

- ♦ Private keys: $2n^2$ bits, Public keys: $2n^2$ bits, Signature size: n^2 bits.
- ♦ For $n = 256$, these are 16,384, 16,384, and 8,192 bytes.
- ♦ **Solution:** The private key components $x_{i,j}$ are generated from a randomly-generated n -bit seed and a counter using a pseudorandom function: $x_{i,j} = \text{prf}(\text{SEED}, i, j)$.
- ♦ The seed is securely stored, and used to generate the $x_{i,j}$ when needed.
- ♦ The private key size is reduced to n bits, or 32 bytes when $n = 256$.



Problem #2: Large public keys and signatures

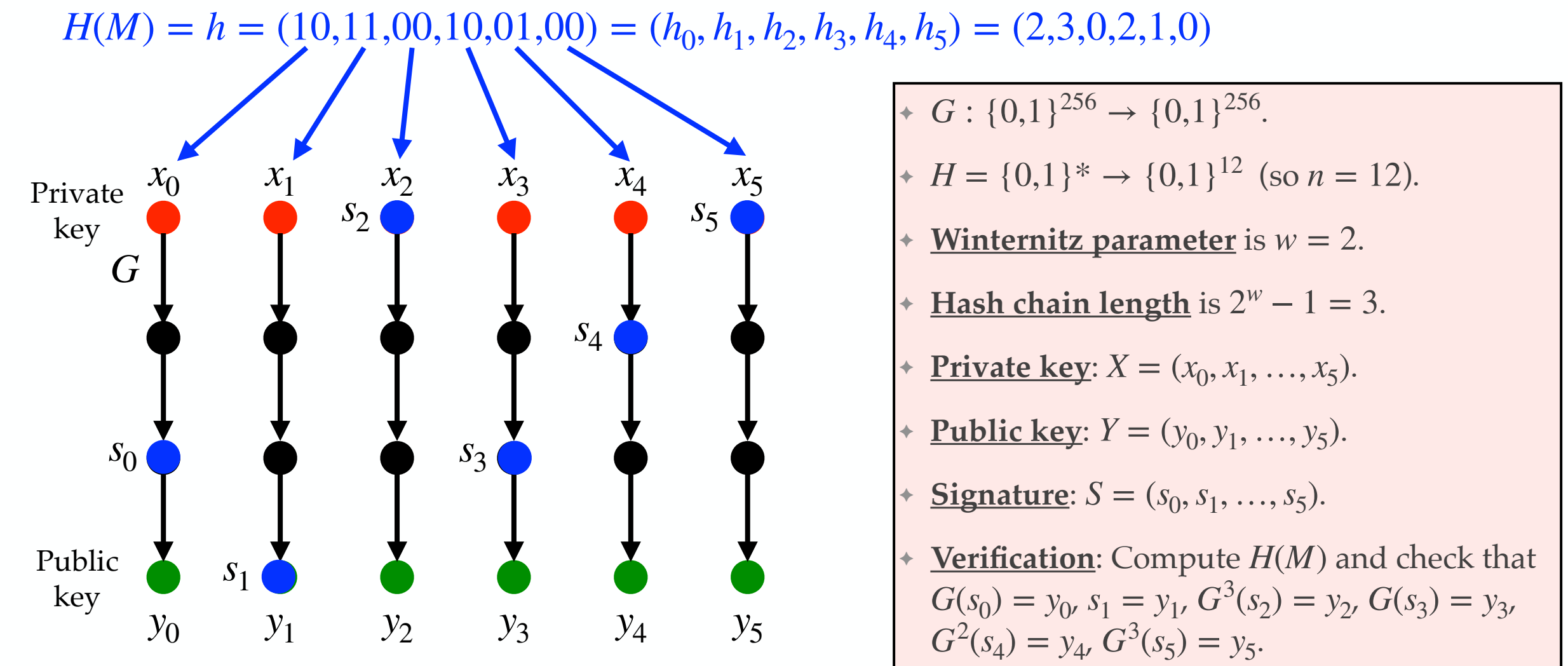
- ♦ Public keys are $2n^2$ bits in size, signatures are n^2 bits in size.
- ♦ The large sizes are because the bits of $H(M)$ are signed one bit at a time.
- ♦ **Solution:** (Winternitz, 1979) Sign the bits of $H(M)$, one w -bit block at a time using *hash chains*.



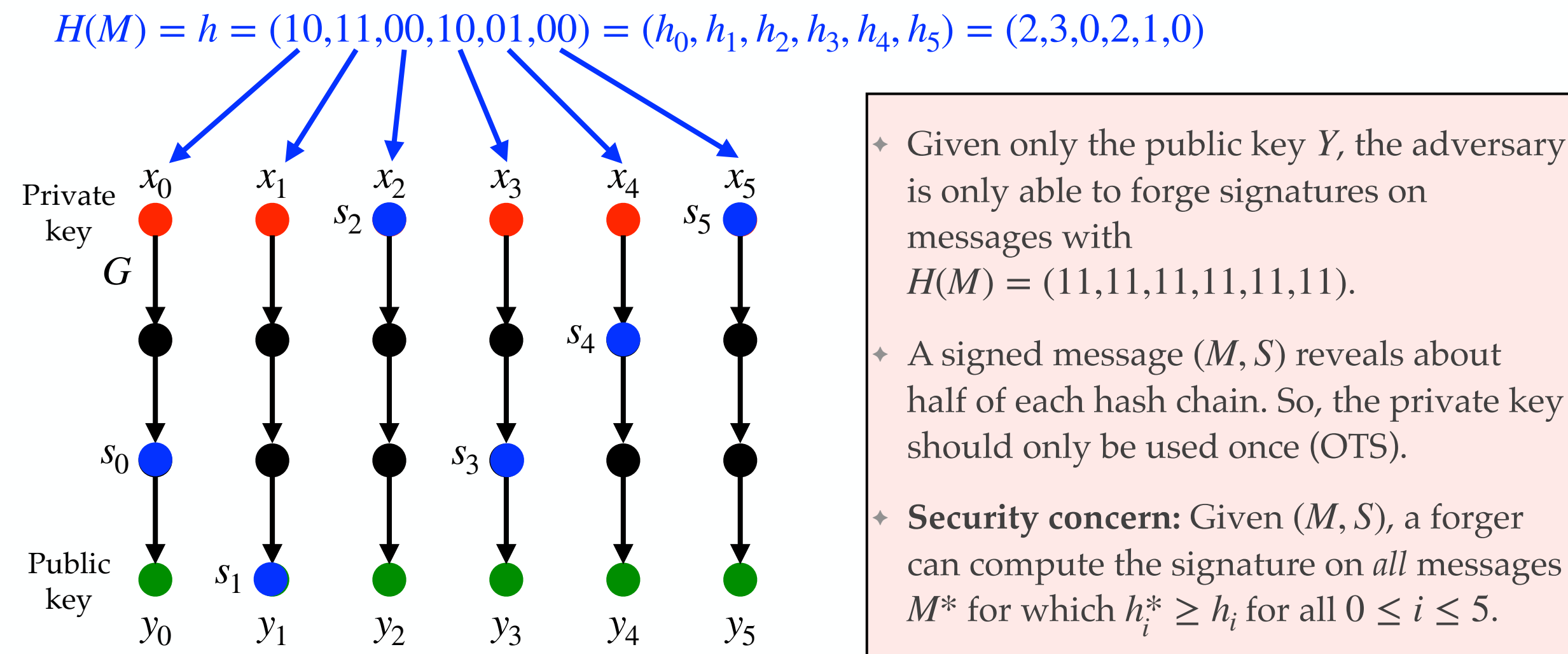
Hash chains

- ✦ $G : \{0,1\}^n \rightarrow \{0,1\}^n$ is a preimage-resistant hash function.
 - ✦ Let $x_0 \in_R \{0,1\}^n$.
 - ✦ Define $x_i = G(x_{i-1})$ for $i \geq 1$, so $x_i = G^i(x_0)$.
 - ✦ The sequence $x_0, x_1, x_2, \dots, x_m$ is called a **hash chain of length m** .
- 
- ✦ Given any x_i , it's easy to compute x_j for all $j > i$.
 - ✦ However, computing x_j for any $j < i$ is infeasible since G is preimage resistant.

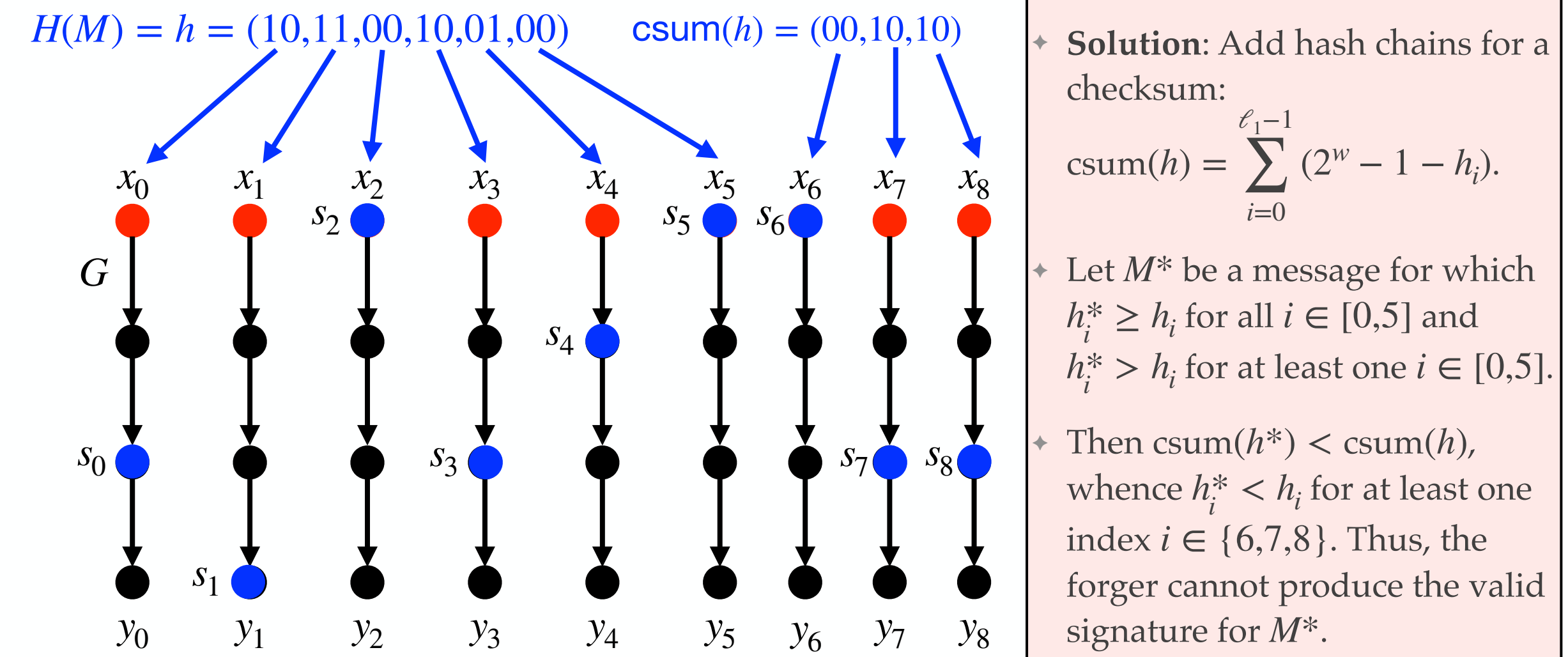
Winternitz OTS: Toy example (1)



Winternitz OTS: Toy example (2)



Winternitz OTS: Toy example (3)



- ♦ $G : \{0,1\}^n \rightarrow \{0,1\}^n$ is preimage resistant, $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is collision resistant.
- ♦ **Parameters:** w (divisor of n), $\ell_1 = n/w$ (# of w -bit blocks in h), $\ell_2 = \lfloor \log_2(\ell_1(2^w - 1))/w \rfloor + 1$ (# of w -bit blocks in $\text{csum}(h)$), $\ell = \ell_1 + \ell_2$.
- ♦ **Key generation:** Alice selects $x_i \in_R \{0,1\}^n$ and computes $y_i = G^{2^w-1}(x_i)$, $0 \leq i \leq \ell - 1$. Her **private key** is $X = (x_0, \dots, x_{\ell-1})$; **public key** is $Y = (y_0, \dots, y_{\ell-1})$.
- ♦ **Signature generation:** Compute $h = H(M) = (h_0, \dots, h_{\ell_1-1})$ and $\text{csum}(h) = (h_{\ell_1}, \dots, h_{\ell-1})$, where each h_i is a w -bit block. Alice's signature on M is $S = (s_0, \dots, s_{\ell-1})$ where $s_i = G^{h_i}(x_i)$ for $0 \leq i \leq \ell - 1$.
- ♦ **Signature verification:** Compute $h = H(M) = (h_0, \dots, h_{\ell_1-1})$ and $\text{csum}(h) = (h_{\ell_1}, \dots, h_{\ell-1})$, and verify that $G^{2^w-1-h_i}(s_i) = y_i$ for $0 \leq i \leq \ell - 1$.

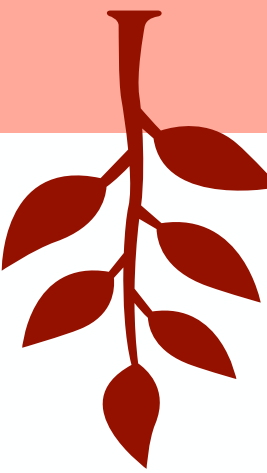
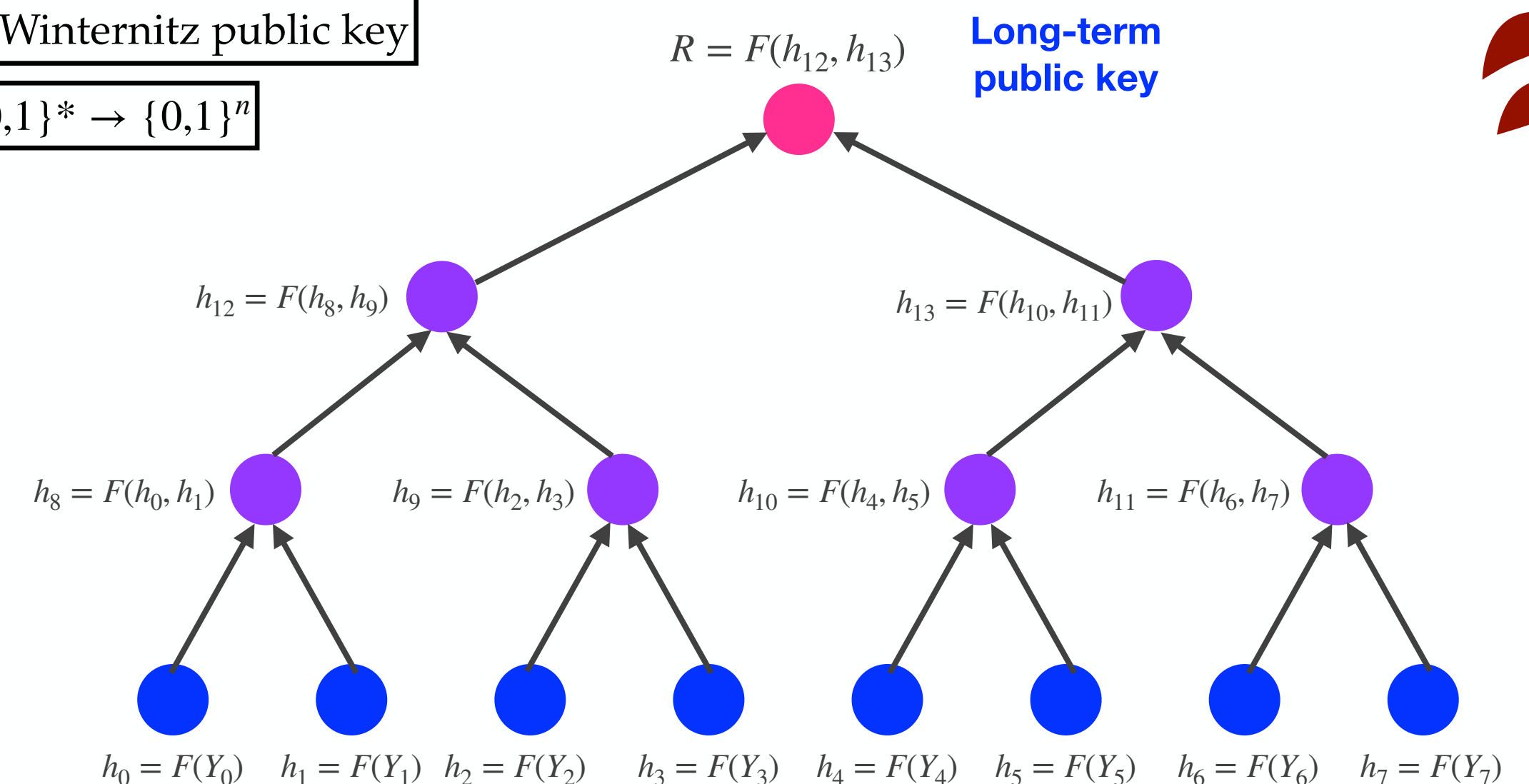
- ♦ Suppose that $n = 256$ and $w = 8$ (so a hash chain has length 255).
- ♦ Then $\ell_1 = 32$, $\ell_2 = 2$, and $\ell = \ell_1 + \ell_2 = 34$.
- ♦ **Public key size:** 1,088 bytes (vs. 16,384 bytes for Lamport).
- ♦ **Signature size:** 1,088 bytes (vs. 8,192 bytes for Lamport).
- ♦ **Private key size:** 32 bytes (using a secret seed and prf).

Problem #3: A private key can only be used once

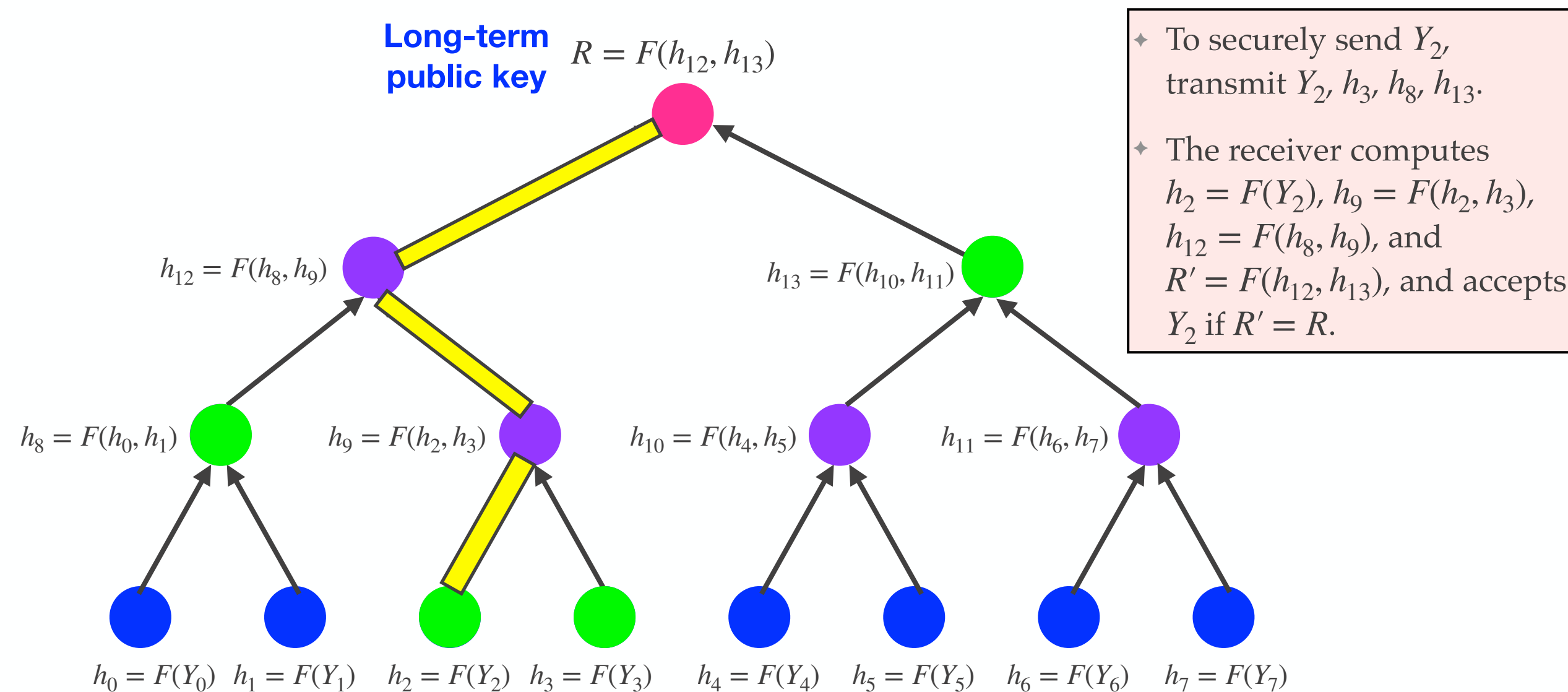
- ♦ **Solution:** Generate many Winternitz OTS key pairs, and use them one at a time.
- ♦ **Problem:** Need a large number of OTS keys if you expect to sign a large number of messages. The total size of all the OTS public keys is very large.
- ♦ **Solution:** Use a **Merkle tree** to authenticate the OTS public keys.
- ♦ **Main idea:** Build a complete binary tree whose leaves are assigned the hashes of the OTS public keys. Each internal node's value is obtained by hashing the concatenation of the values of its two children. The user's **long-term public key** is the value assigned to the root of the tree. **Authentication paths** to the root are used to confirm the authenticity of an individual OTS public key.

$Y_i =$ Winternitz public key

$F : \{0,1\}^* \rightarrow \{0,1\}^n$



Merkle trees: Authentication paths



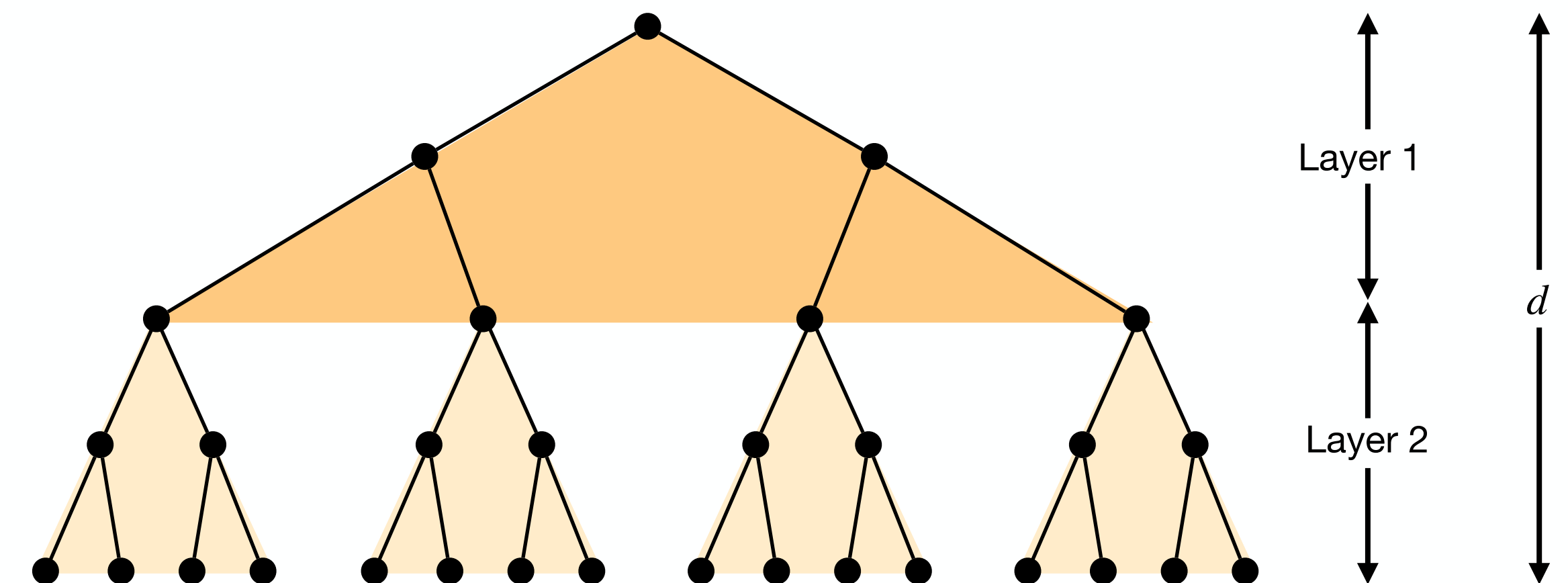
Example: Winternitz OTS + Merkle trees

- ♦ **Winternitz parameters:** $n = 256, w = 8, \ell = 34$.
- ♦ **Merkle tree:** Height 20 (so 2^{20} Winternitz OTS key pairs).
- ♦ **Long-term public key size:** 32 bytes.
- ♦ **Private key size:** 32 bytes (using a secret seed and prf).
- ♦ **Signature size:** $1,088 + 1,088 (Y_i) + 20 \times 32$ (auth. path) = 2,816 bytes.

Problem #4: Constructing a large Merkle tree is slow

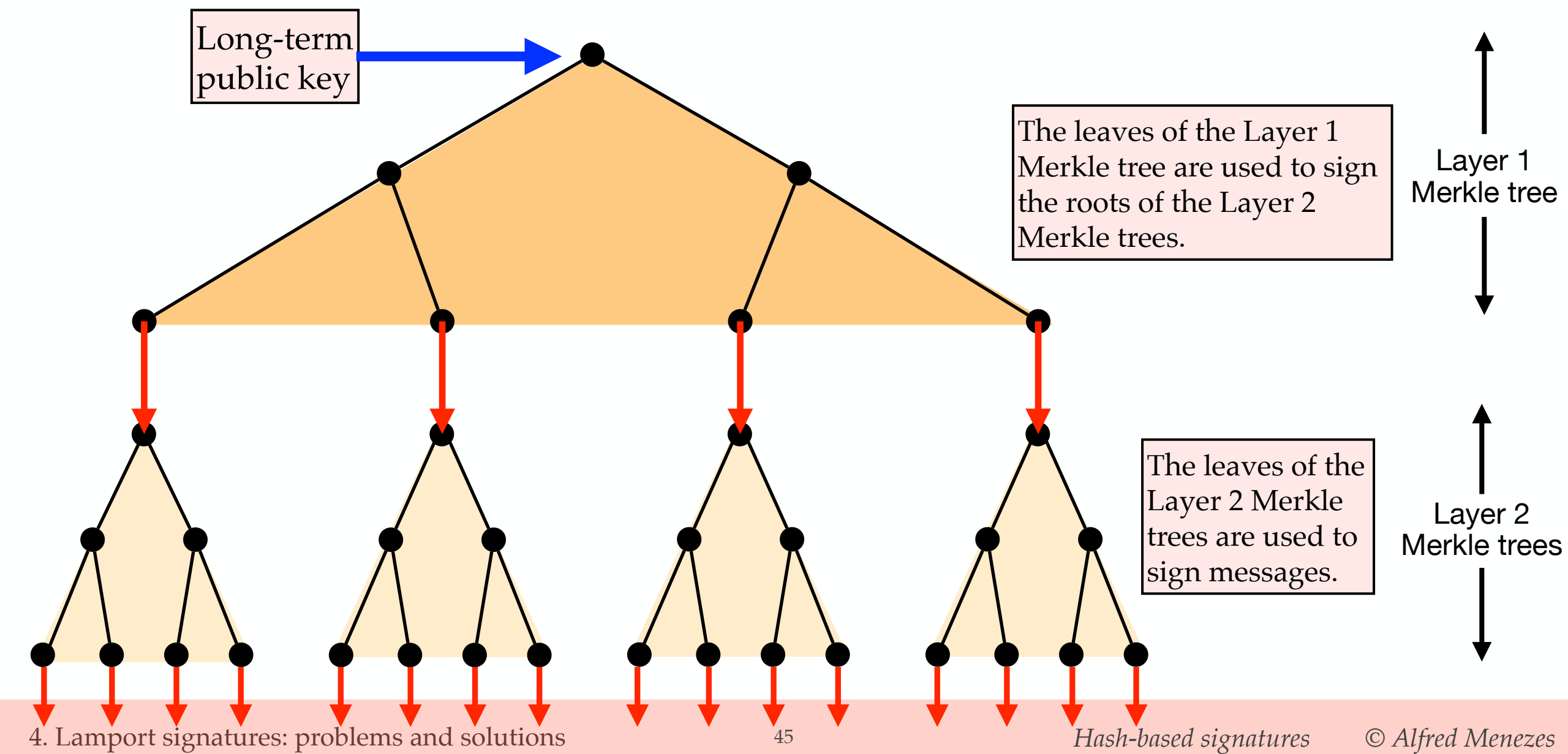
- ♦ To support a large number of signatures, the Merkle tree must have large height d .
- ♦ For example, to support 2^{40} signatures, the Merkle has height 40.
- ♦ However, key generation is very slow since all 2^d OTS key pairs need to be generated in order to determine the root R (the long-term public key) of the tree.
- ♦ **Solution:** The Merkle tree is divided into smaller trees to form a *hypertree*, also called a *multi-tree*.

Hypertrees (1)



Of course, more than two layers can be used.

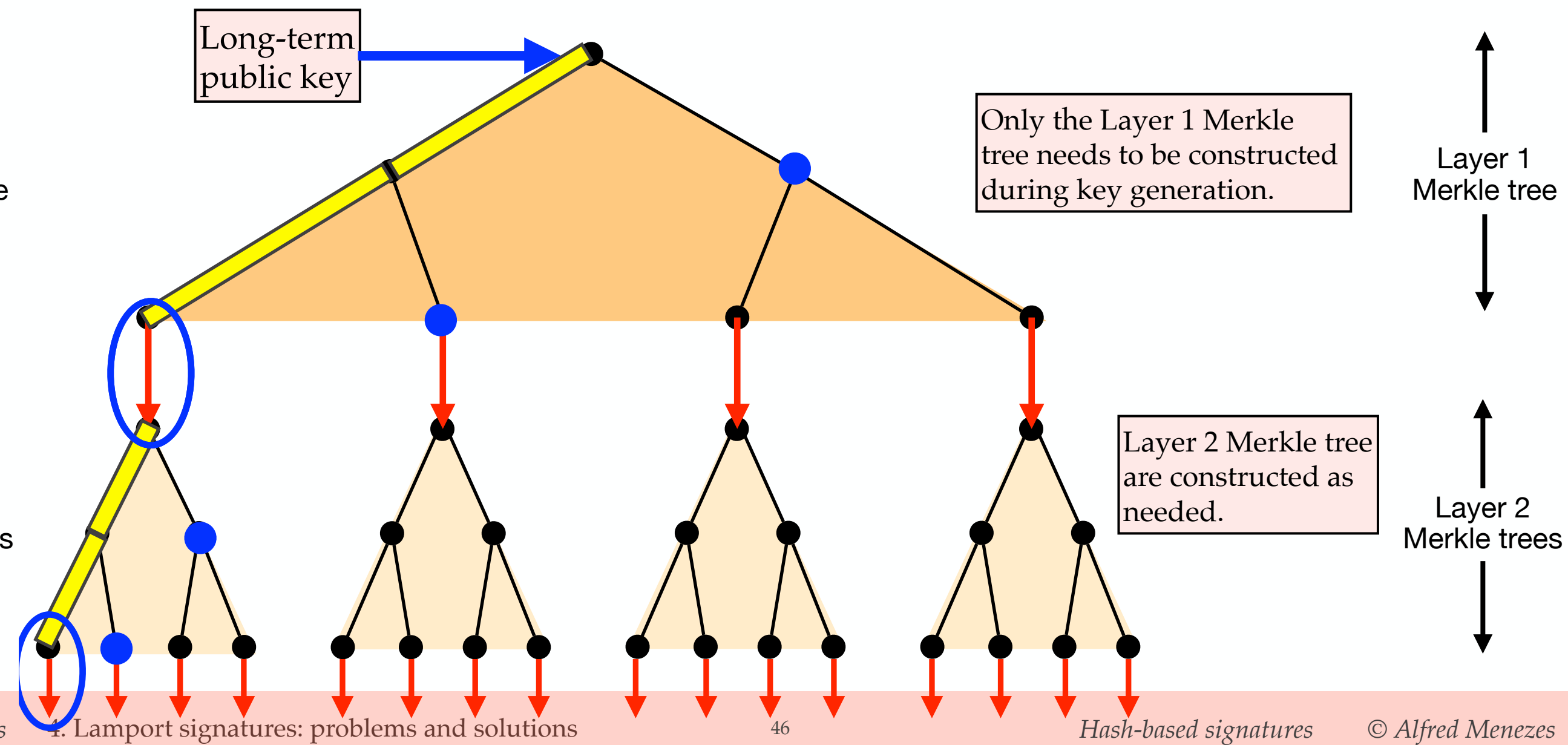
Hypertrees (2)



LMS signature scheme

- ♦ The **Leighton-Micali signature scheme (LMS)** uses the Winternitz OTS with Merkle trees (and hypertrees), and some additional optimizations to enhance security.
- ♦ LMS will be presented in video V5.

Using a hypertree



HASH-BASED SIGNATURES

5. THE LEIGHTON-MICALI SIGNATURE SCHEME

Alfred Menezes
cryptography101.ca

1. Introduction
2. LM-OTS description
3. LMS description
4. HSS
5. XMSS
6. Stateful signature schemes

- ✦ The Leighton-Micali signature scheme (LMS) is a hash-based signature scheme standardized by the IETF (RFC 8554) in 2019.
- ✦ LMS is fully described in RFC 8554.
 - ✦ **LM-OTS**: One-time signature scheme
 - ✦ **LMS**: LM-OTS + Merkle trees
 - ✦ **HSS** (Hierarchical signature scheme): LMS + hypertrees.
- ✦ LMS was also adopted as a NIST standard (SP 800-208) in 2020.
 - ✦ SP 800-208 points to RFC 8554, and added some parameter sets.
- ✦ **XMSS** (eXtendable Merkle Signature Scheme) is another hash-based signature scheme, standardized by the IETF (RFC 8391) in 2018, and by NIST in 2020 (SP 800-28).



LM-OTS parameters

- ✦ An n -bit hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$.
 - ✦ For concreteness, we'll take $n = 256$ and $H = \text{SHA256}$.
- ✦ **Winternitz parameter** w , a divisor of n .
 - ✦ SP 800-208 permits $w \in \{1, 2, 4, 8\}$.
- ✦ $\ell_1 = n/w$ (number of w -bit blocks in a hash value).
- ✦ $\ell_2 = \lfloor \log_2(\ell_1(2^w - 1)) \rfloor + 1$ (number of w -bit blocks in a checksum).
 - ✦ The **checksum** $\text{csum}(h)$ of a hash value $h = (h_0, h_1, \dots, h_{\ell_1-1})$, where each h_i is a w -bit block, is

$$\text{csum}(h) = \sum_{i=0}^{\ell_1-1} (2^w - 1 - h_i).$$
- ✦ $\ell = \ell_1 + \ell_2$.
 - ✦ $\ell = 265, 133, 67, 34$ for $w = 1, 2, 4, 8$ (and $n = 256$).

LM-OTS is essentially the Winternitz-OTS with some modifications.

LM-OTS modifications (1)

- ✦ The **public key** is $K = H(y_0, y_1, \dots, y_{\ell-1})$ instead of $Y = (y_0, y_1, \dots, y_{\ell-1})$.
- ✦ A **single hash function** $H : \{0,1\}^* \rightarrow \{0,1\}^n$ is used, and also shared with LMS and HSS.
- ✦ A **unique prefix** is appended before hashing. The prefix includes:
 - ✦ A randomly-selected 32-byte identifier I of the Merkle tree associated with the LM-OTS instance.
 - ✦ A 4-byte number q that identifies the tree's leaf associated with the LM-OTS instance.
 - ✦ The index i of the w -bit block h_i within the message hash / checksum, which determines which hash chain to use
 - ✦ The position j within a Winternitz hash chain.
 - ✦ A two byte constant, 0x8080 (for computing K) or 0x8181 (for hashing a message).

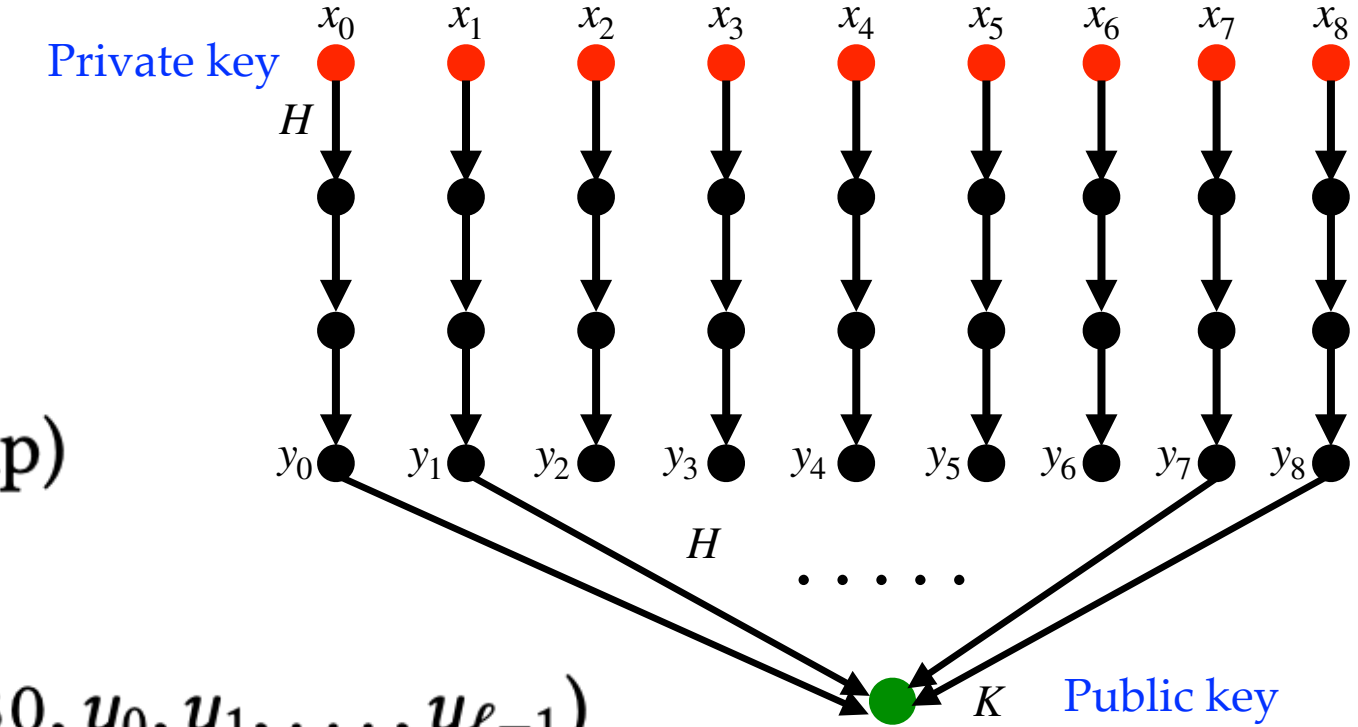
LM-OTS modifications (2)

- ♦ A randomly-generated 256-bit string C is appended to the message M before hashing: $h = H(C, M)$.
 - ♦ C is called the *message randomizer*.
 - ♦ C is included with the signature, since it's needed for signature verification.
- ♦ This eliminates the need for collision resistance for H (since C is randomly chosen by the signer each time a message is signed).

LM-OTS key generation

Key generation. Alice generates a public-private key pair as follows:

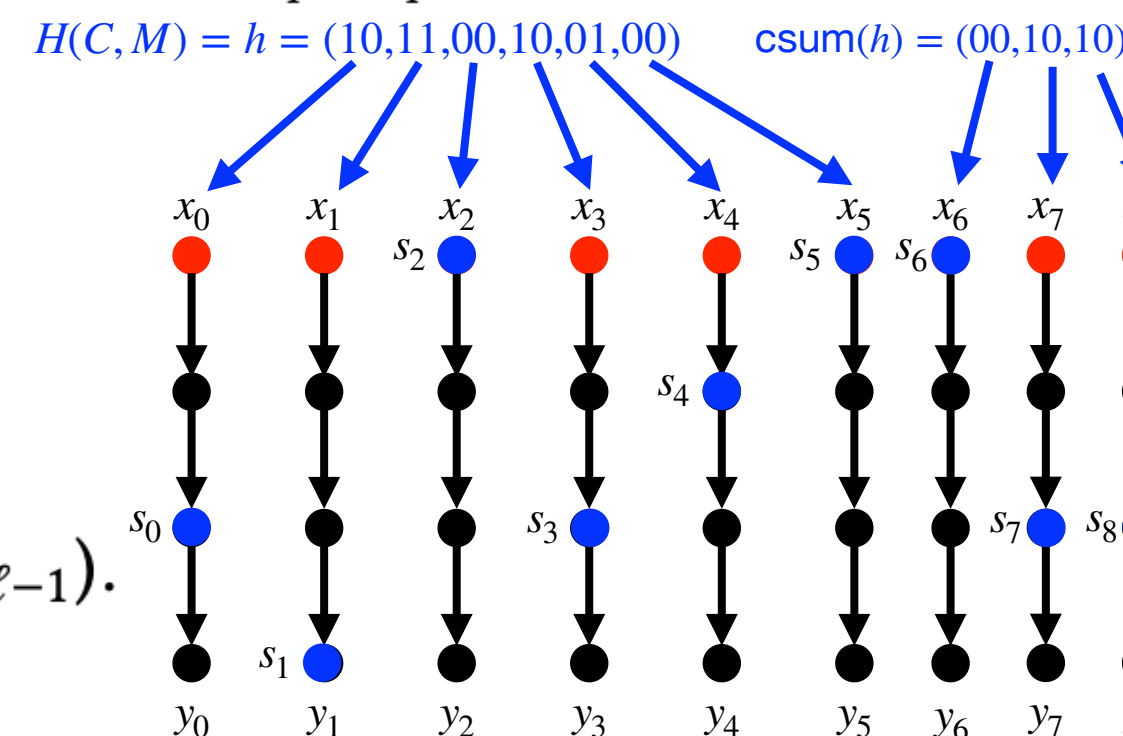
- 1 **for** $i = 0$ **to** $\ell - 1$ **do**
- 2 Select $x_i \in_R \{0, 1\}^n$
- 3 $\text{tmp} \leftarrow x_i$
- 4 **for** $j = 0$ **to** $2^w - 2$ **do**
- 5 $\text{tmp} \leftarrow H(I, q, i, j, \text{tmp})$
- 6 $y_i \leftarrow \text{tmp}$
- 7 Compute $K = H(I, q, 0x8080, y_0, y_1, \dots, y_{\ell-1})$
- 8 Alice's private key is $X = (x_0, x_1, \dots, x_{\ell-1})$; her public key is (I, q, K)



LM-OTS signature generation

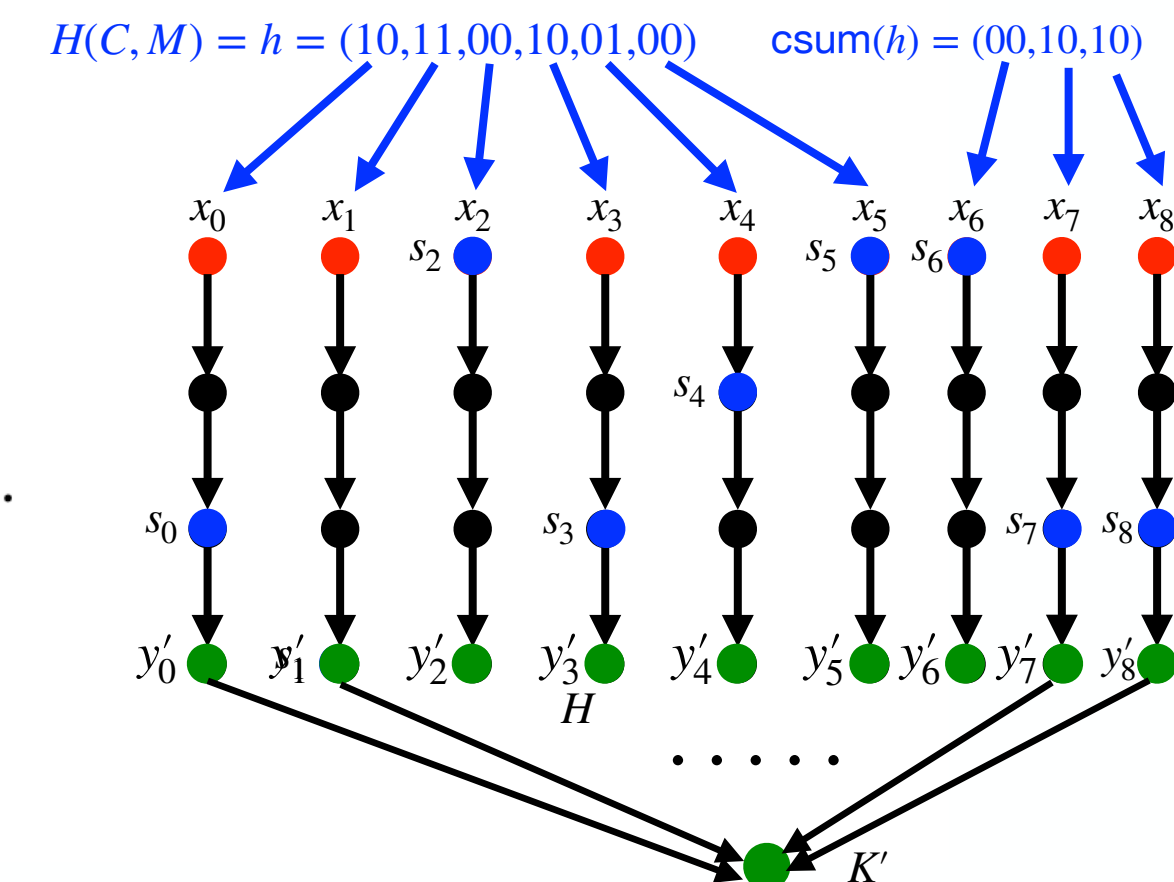
Signature generation. To sign a message $M \in \{0, 1\}^*$, Alice does the following:

- 1 Select $C \in_R \{0, 1\}^n$.
- 2 Compute $h = H(I, q, 0x8181, C, M)$; the w -bit blocks of h are $(h_0, h_1, \dots, h_{\ell-1})$.
- 3 Compute $\text{csum}(h)$ and obtain the w -bit blocks $(h_{\ell_1}, h_{\ell_1+1}, \dots, h_{\ell-1})$ of $\tilde{c}(h)$.
- 4 **for** $i = 0$ **to** $\ell - 1$ **do**
- 5 $\text{tmp} \leftarrow x_i$
- 6 **for** $j = 0$ **to** $h_i - 1$ **do**
- 7 $\text{tmp} \leftarrow H(I, q, i, j, \text{tmp})$
- 8 $s_i \leftarrow \text{tmp}$
- 9 Alice's signature on M is $(C, s_0, s_1, \dots, s_{\ell-1})$.



Signature verification. To verify Alice's signature $(C, s_0, \dots, s_{\ell-1})$ on M , Bob does:

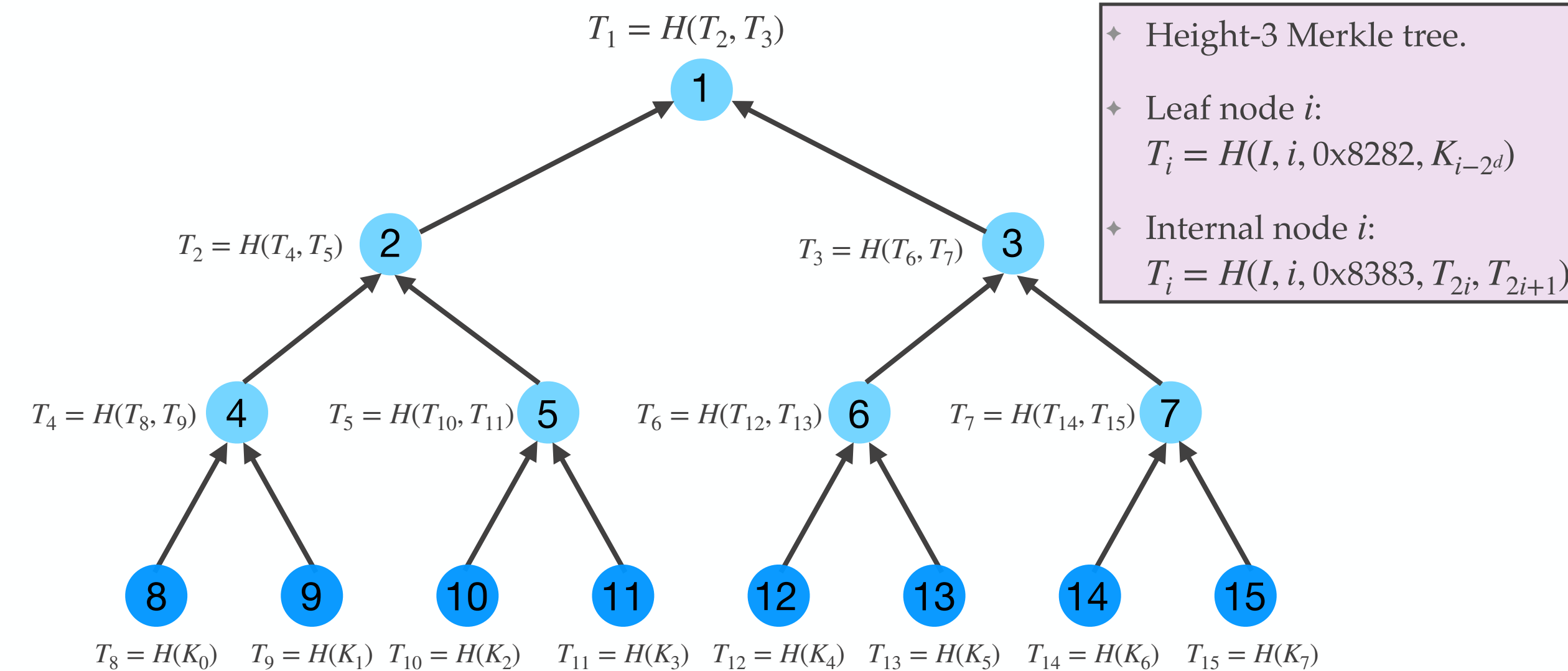
- 1 Compute $h = H(I, q, 0x8181, C, M)$; the w -bit blocks of h are $(h_0, h_1, \dots, h_{\ell-1})$.
- 2 Compute $\text{csum}(h)$ and obtain the w -bit blocks $(h_{\ell_1}, h_{\ell_1+1}, \dots, h_{\ell-1})$ of $\tilde{c}(h)$.
- 3 **for** $i = 0$ **to** $\ell - 1$ **do**
- 4 $\text{tmp} \leftarrow s_i$
- 5 **for** $j = h_i$ **to** $2^w - 2$ **do**
- 6 $\text{tmp} \leftarrow H(I, q, i, j, \text{tmp})$
- 7 $y'_i \leftarrow \text{tmp}$
- 8 Compute $K' = H(I, q, 0x8080, y'_0, y'_1, \dots, y'_{\ell-1})$.
- 9 **if** $K' = K$ **then**
- 10 **return** Accept
- 11 **else**
- 12 **return** Reject



LMS

- ✦ LMS uses a height- d Merkle tree, whose leaves are associated with LM-OTS public keys $K_0, K_1, \dots, K_{2^d-1}$.
- ✦ SP 800-208 permits heights $d \in \{5, 10, 15, 20, 25\}$.
- ✦ Private keys are generated from a 256-bit random secret SEED:
 - ✦ The i th component of private key associated with a leaf numbered q is $H(I, q, i, 0xff, \text{SEED})$.

LMS tree

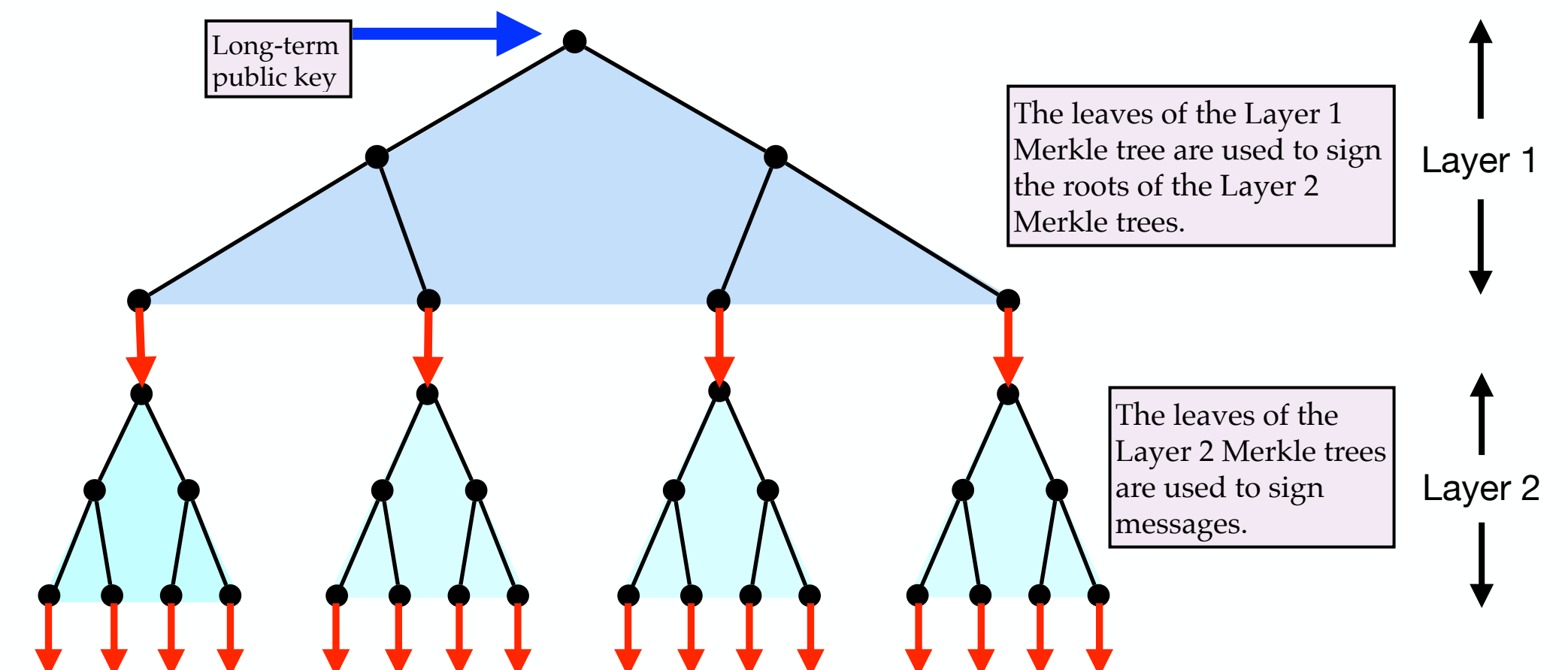


LMS security

- ✦ (Katz, 2016) LMS has been proven secure in the random oracle model, wherein the hash function H is treated as a public, randomly-selected function.
- ✦ (Eaton, 2018) LMS has also been proven secure in the quantum random oracle model, wherein adversaries can query H in superposition.

HSS

- ✦ **Hierarchical Signature Scheme (HSS)** is the hypertree version of LMS.
- ✦ SP 800-208 permits hypertrees with as many as 8 layers.



- ✦ **XMSS** closely resembles LMS.
- ✦ The main difference is its use of “**bitmasks**”, which are pseudorandom bit strings that are XORed with inputs to the hash function.
- ✦ The bitmasks enable security proofs for XMSS in the “standard model”, i.e., without needing to model the hash function as a random oracle.
- ✦ The hypertree version of XMSS is called **XMSS^{MT}**.
- ✦ XMSS and XMSS^{MT} are fully specified in IETF RFC 8391, and additional parameter sets are defined in NIST’s SP 800-208.

- ✦ LMS and XMSS are **stateful** signature schemes.
- ✦ A signer must be very careful not to reuse any of their OTS private keys associated with their long-term public key.
- ✦ To enforce this requirement, the signer could keep track of a counter or key index that increments after each message is signed.
 - ✦ This type of data that changes over time is called **state**.
- ✦ However, maintaining state securely in practice can be challenging.
 - ✦ e.g., if a system crashes and is restored to an earlier state.

HASH-BASED SIGNATURES

6. SPHINCS+

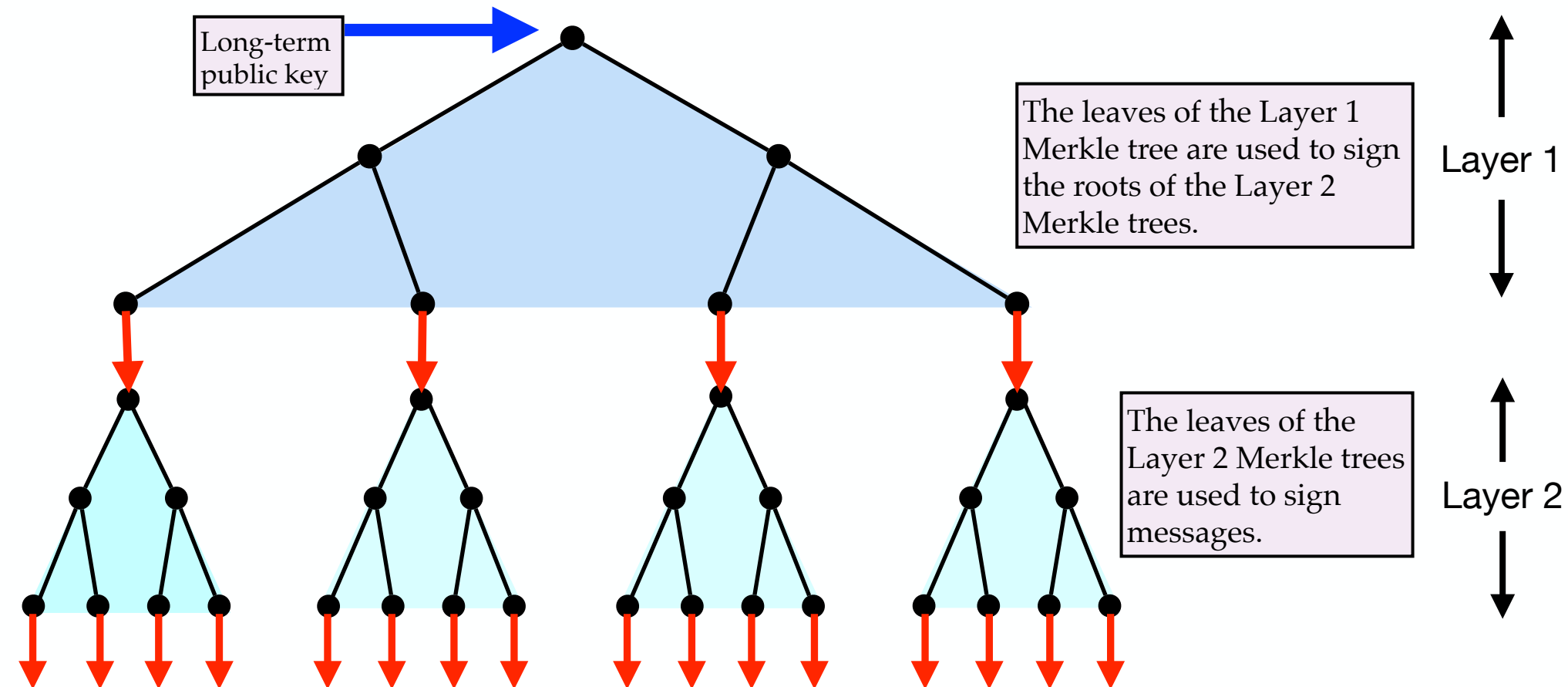
Alfred Menezes
cryptology101.ca

Outline

1. Stateful signature schemes
2. SPHINCS+: A stateless signature scheme
3. Few-times signature schemes
4. Virtual hypertrees

Stateful signature schemes

- ♦ NIST's SP 800-208: LMS and HSS, XMSS and XMSS^{MT}.



SPHINCS+: A stateless signature scheme

- ♦ Designed by Bernstein, Hülsing, Kölbl, Niederhagen, Rijneveld and Schwabe.
- ♦ Standardized by NIST in August 2024: **FIPS 205**.
 - ♦ Official name is **SLH-DSA** (Stateless Hash-based Digital Signature Algorithm).
- ♦ Main idea:
 - ♦ Employ a hypertree with a *very large* number of leaves.
 - ♦ The leaves represent signature key pairs for an OTS.
 - ♦ To sign a message, the signer *randomly* selects one of these key pairs.
 - ♦ A limit is placed on the total number of messages that can be signed, ensuring that the probability of key reuse is negligibly small.
- ♦ SPHINCS+ employs numerous optimizations. I'll present the two main optimizations:
 - ♦ **Few-times signatures** and **virtual hypertrees**.

Few-times signature schemes

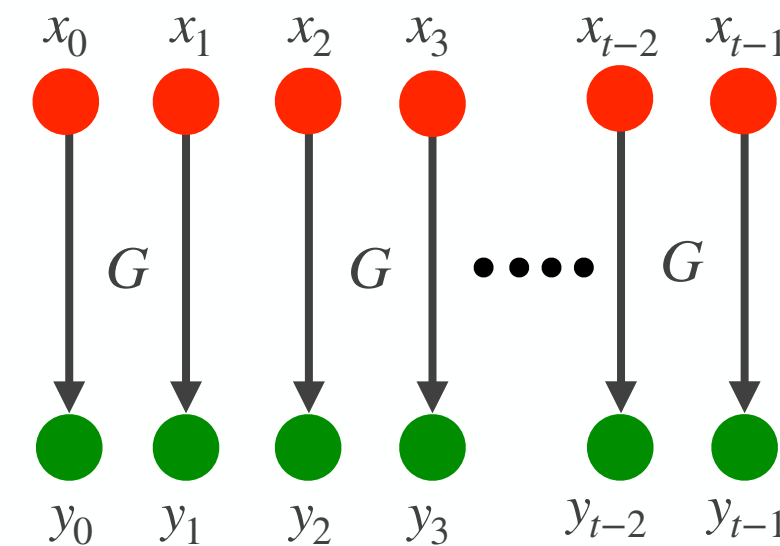
- ♦ Unlike a one-time signature scheme, a **few-times signature scheme (FTS)** can be used a small number of times (e.g., 5-10 times) before security degrades.
- ♦ Main idea: Each signature reveals only a small fraction of the private key components, unlike Lamport's OTS where a single signature reveals half of the private key.
- ♦ Example of a few-times signature scheme: **HORS**.

HORS

- ♦ **HORS**: Hashing to Obtain Random Subset.
- ♦ Proposed by Reyzin and Reyzin in 2002.
- ♦ Ingredients: A preimage resistant hash function $G : \{0,1\}^n \rightarrow \{0,1\}^n$ and a second-preimage resistant hash function $H : \{0,1\}^* \rightarrow \{0,1\}^n$.
- ♦ Parameters: A divisor k of n , $m = n/k$, $t = 2^m$.
- ♦ Sample parameters: $n = 256$, $k = 16$, $m = 16$, $t = 2^{16} \approx 65,000$.

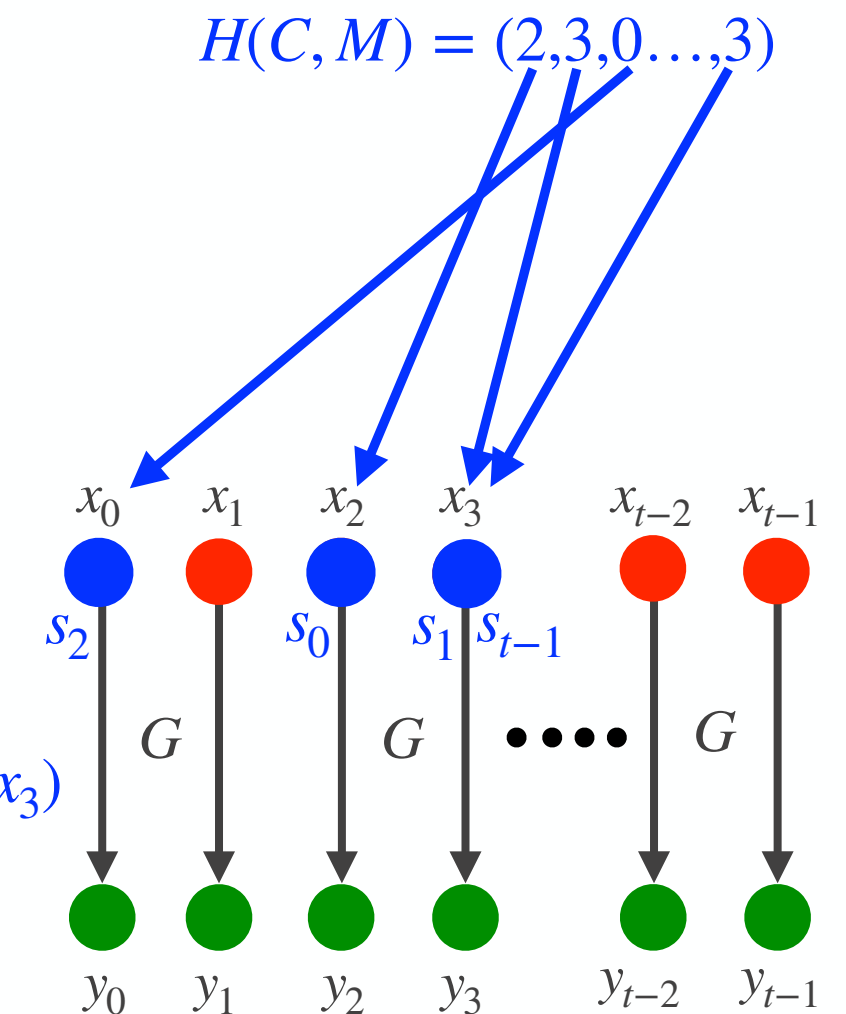
HORS: Key generation

- ♦ Alice selects a key pair as follows:
 - ♦ Select $x_i \in_R \{0,1\}^n$, for $0 \leq i \leq t-1$.
 - ♦ Compute $y_i = G(x_i)$, for $0 \leq i \leq t-1$.
 - ♦ Alice's **private key** is $X = (x_0, x_1, \dots, x_{t-1})$.
 - ♦ Her **public key** is $Y = (y_0, y_1, \dots, y_{t-1})$.



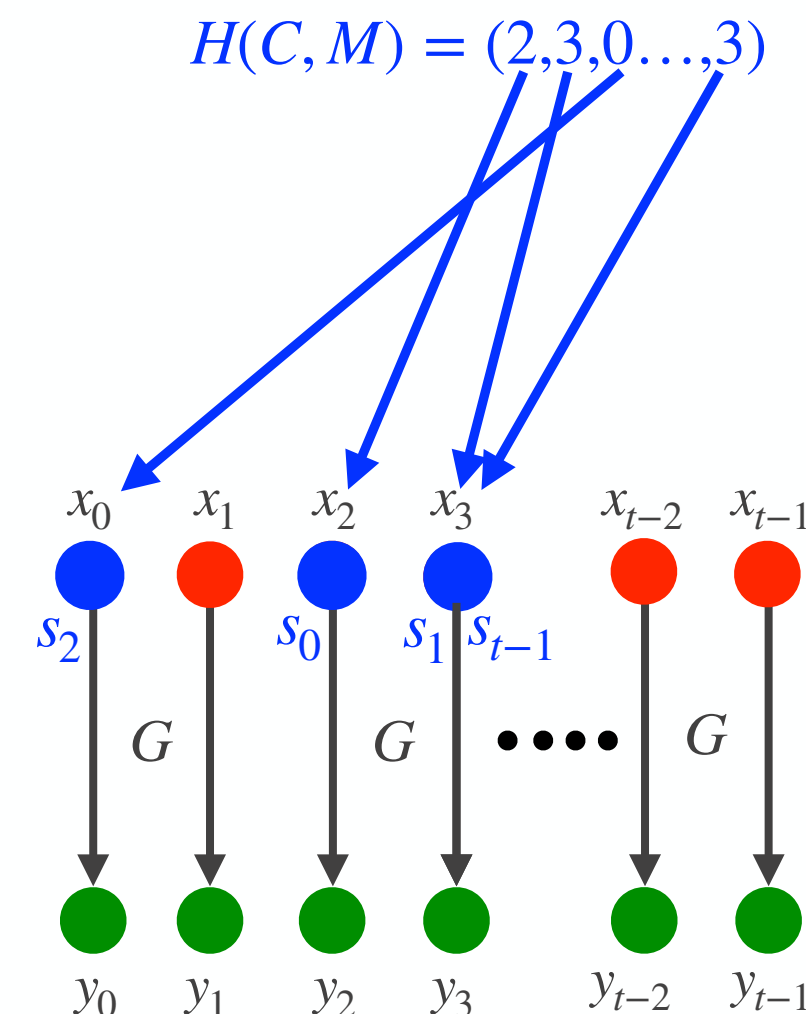
HORS: Signature generation

- ♦ To sign a message $M \in \{0,1\}^*$, Alice does the following:
 - ♦ Select a message randomizer $C \in_R \{0,1\}^n$.
 - ♦ Compute $h = H(C, M)$.
 - ♦ Write $h = (h_0, h_1, \dots, h_{k-1})$, where each $h_i \in \{0,1\}^m$.
 - ♦ Set $S = (s_0, s_1, \dots, s_{k-1})$, where $s_j = x_{h_j}$.
 - ♦ Alice's signature on M is (C, S) .
- ♦ Note: A signature exposes at most k of the t components of X .



HORS: Signature verification

- ♦ To verify Alice's signature (C, S) on M , Bob does:
 - ♦ Obtain an authentic copy of Alice's public key Y .
 - ♦ Compute $h = H(C, M)$.
 - ♦ Write $h = (h_0, h_1, \dots, h_{k-1})$, where each $h_i \in \{0,1\}^m$.
 - ♦ Accept iff $G(s_j) = y_{h_j}$ for all $0 \leq j \leq k-1$.

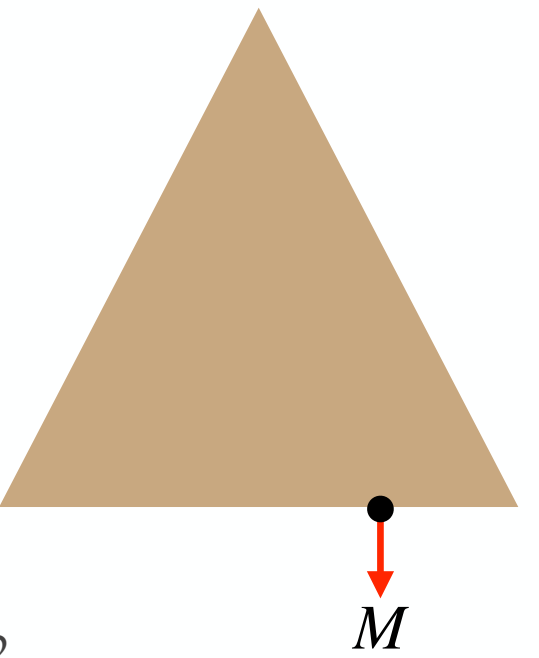


HORS: Security analysis (1)

- ♦ Suppose the adversary has obtained c signed messages $(M_i, (C_i, S_i))$.
- ♦ Since the randomizers C_i were chosen by Alice, the adversary has no control over the hash values $h^{(i)} = H(C_i, M_i)$.
- ♦ Assuming that H behaves like a random function, the c signed messages reveal at most ck randomly-selected components of X .
- ♦ To forge Alice's signature, the adversary needs to find a message M^* and randomizer C^* such that each $x_{h_j^*}$ (where $h^* = H(C^*, M^*) = (h_0^*, h_1^*, \dots, h_{k-1}^*)$) is among the revealed components of X .
 - ♦ This happens with probability at most $(ck/t)^k$.
- ♦ If this probability is negligible, Alice can securely sign up to c messages.

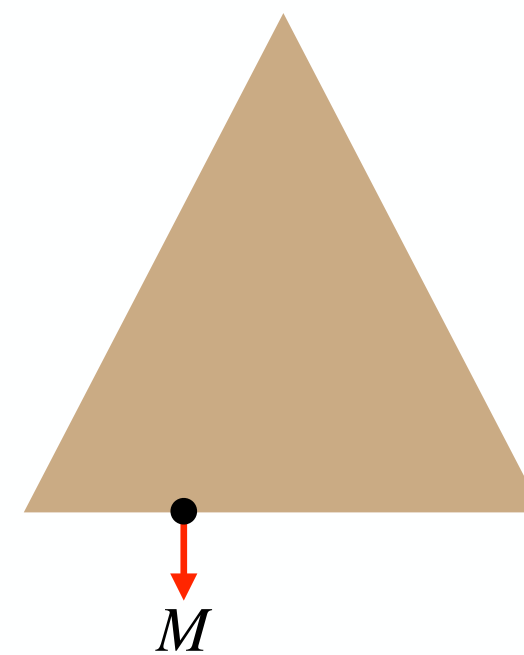
- ✦ Example: Suppose $n = 256$, $k = 16$, $m = 16$, $t = 2^{16}$, and $c = 5$.
 - ✦ Then the probability defined on the previous slide is $(ck/t)^k \approx 2^{-154}$, which is negligible.
- ✦ One drawback: Large public keys (nt bits, or 2.1 Megabytes).
 - ✦ Solution: **HORST** (HORS with Trees).
 - ✦ The t components of Y are assigned to the leaves of a height- m Merkle tree. The root of the tree is Alice's public key.

- ✦ Consider a height- d Merkle tree whose $N = 2^d$ leaves correspond to the public keys of an OTS.
- ✦ To sign a message, a leaf is chosen *at random*.
- ✦ Suppose that a limit of S is placed on the number of messages that can be signed.
- ✦ By the birthday paradox, the probability that any OTS private key is reused for two or messages is at most $\binom{S}{2} \frac{1}{N} \approx \frac{S^2}{2N}$.
- ✦ Example: If $S = 2^{40}$ and $d = 207$, this probability is at most $1/2^{128}$.
- ✦ Problem: Constructing a height-207 Merkle tree is completely impractical.



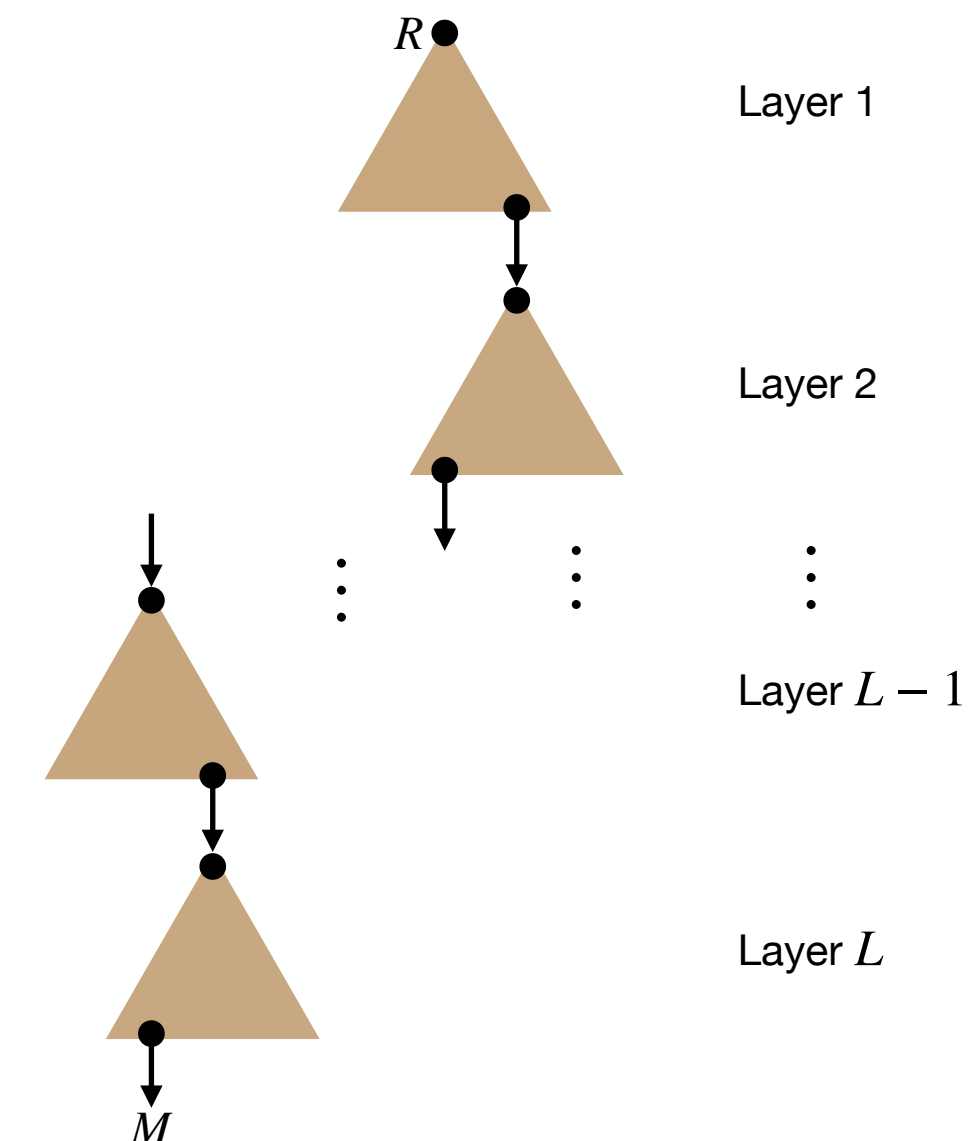
Merkle tree + Few-times signatures

- ✦ Each leaf is associated with a key pair for a FTS instead of an OTS.
- ✦ Messages are signed using a randomly selected leaf's private key.
- ✦ Suppose that a limit of S is placed on the number of messages that can be signed.
- ✦ By a generalization of the birthday paradox for multi-collisions, the probability that any FTS private keys is used more than c times is at most $\binom{S}{c} \frac{1}{N^{c-1}} \approx \frac{S^c}{c! N^{c-1}}$.
- ✦ Example: If $S = 2^{40}$, $c = 5$, and $d = 80$, this probability is at most $1/2^{127}$.
- ✦ Problem: Constructing a height-80 Merkle tree is also impractical.



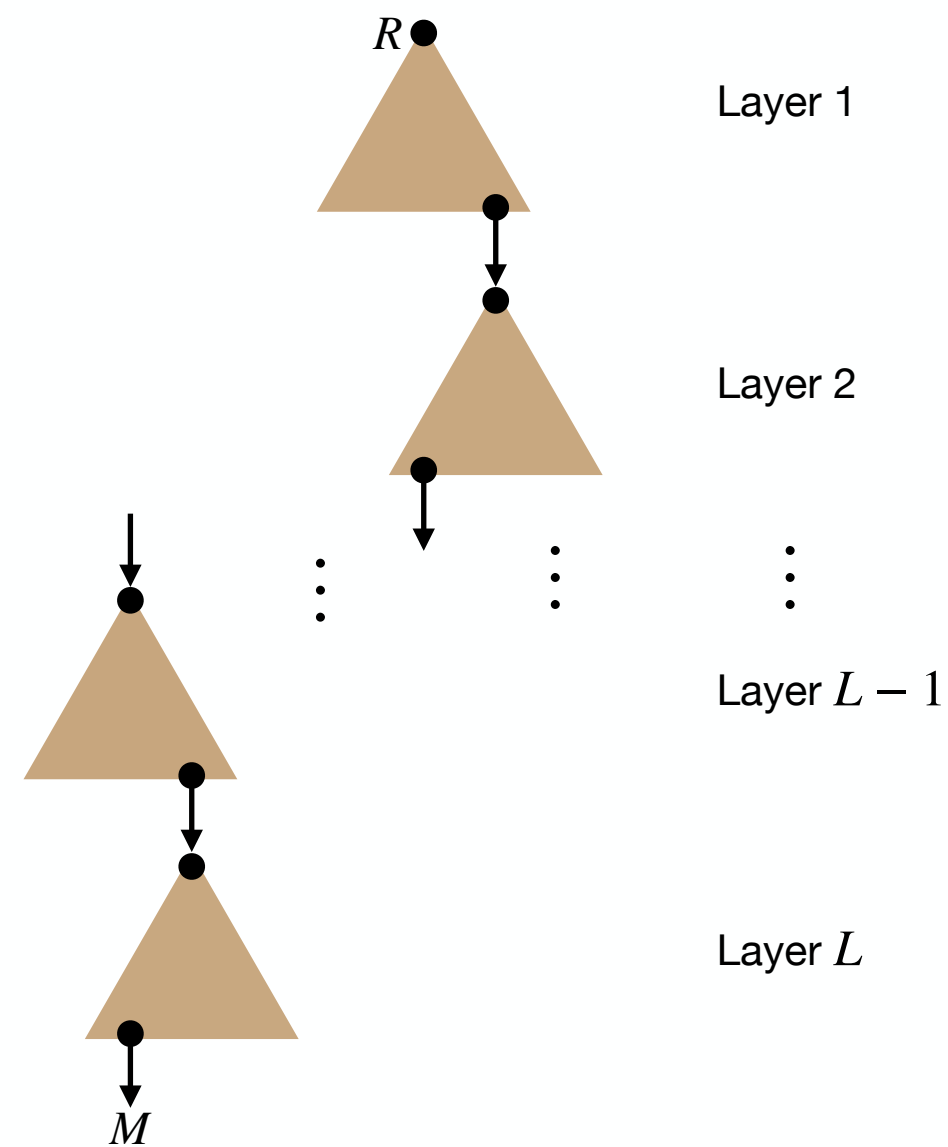
Virtual hypertrees (1)

- ✦ A **virtual hypertree** has L layers.
- ✦ Each layer consists of height- d Merkle trees.
- ✦ Every leaf of the Merkle trees in layers 1 to $L - 1$ is associated with an **OTS key pair**.
 - ✦ Each OTS private key is used to sign the root of the tree beneath it.
- ✦ Every leaf of the Merkle trees in layer L is associated with an **FTS key pair**.
 - ✦ The FTS private keys are used to sign messages.
- ✦ The root R of the hypertree is the user's **long-term public key**.



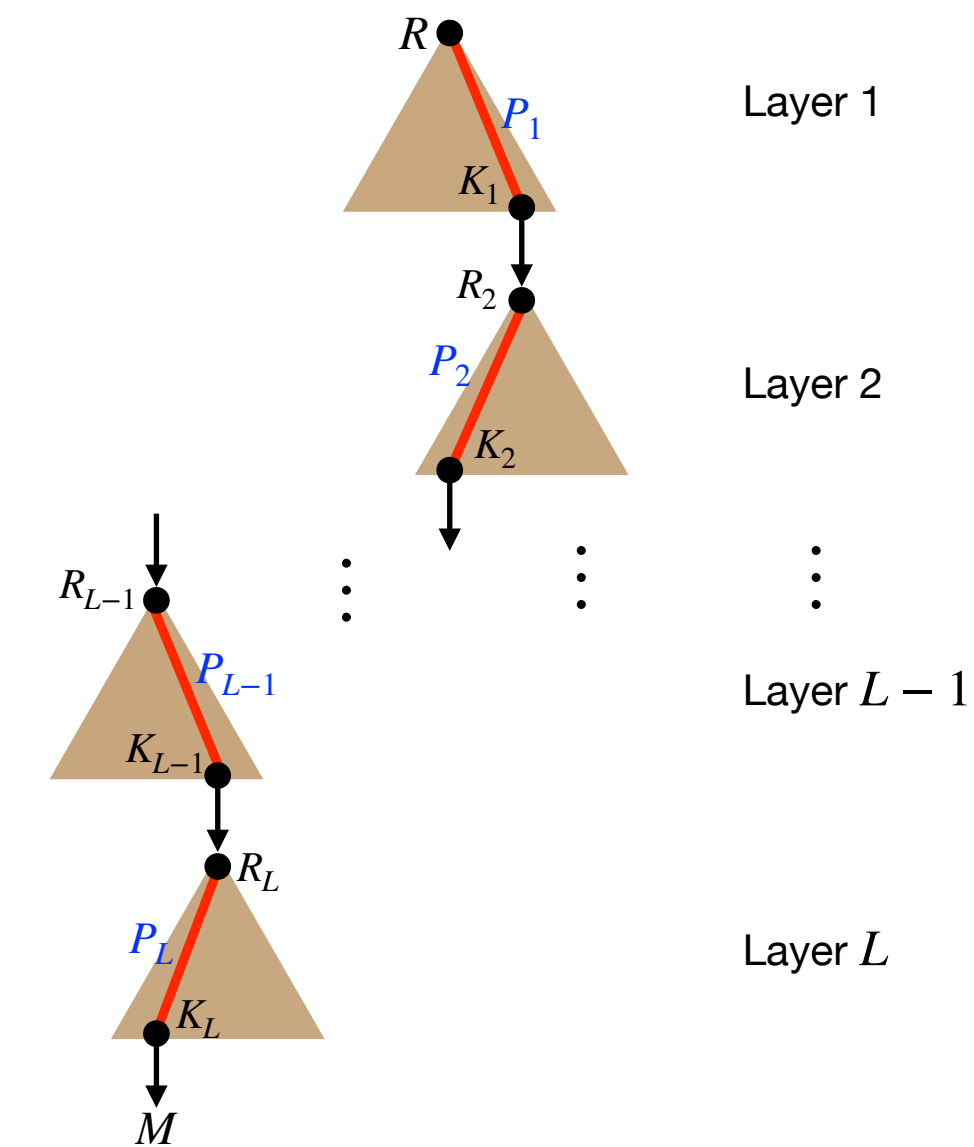
Virtual hypertrees (2)

- ♦ All OTS and FTS private keys are derived by applying a prf to a **single secret SEED** and a unique index identifying the corresponding public key.
 - ♦ The index might include the layer number, the tree identifier within that layer, and the public key's position within that tree.
- ♦ Once SEED is chosen, the entire hypertree is fully defined, even though it might be infeasible to actually construct the entire tree.
 - ♦ This is the sense in which the hypertree is “virtual”.
- ♦ Any individual Merkle tree can be efficiently constructed from SEED.



Signature generation

- ♦ To verify a signed message, Bob first validates the authentication path P_1 from K_1 to R .
- ♦ He then verifies the OTS signature on R_2 using K_1 and validates the authentication path P_2 from K_2 to its Merkle root R_2 .
- ♦ This is continued downward, layer by layer through the hypertree, until reaching K_L .
- ♦ He is then confident of the authenticity of the FTS public key K_L he was sent.
- ♦ Finally, Bob verifies the signature on M using K_L .



Signature verification

- ♦ To verify a signed message, Bob first validates the authentication path P_1 from K_1 to R .
- ♦ He then verifies the OTS signature on R_2 using K_1 and validates the authentication path P_2 from K_2 to its Merkle root R_2 .
- ♦ This is continued downward, layer by layer through the hypertree, until reaching K_L .
- ♦ He is then confident of the authenticity of the FTS public key K_L he was sent.
- ♦ Finally, Bob verifies the signature on M using K_L .

